太原理工大学
TAIYUAN UNIVERSITY OF TECHNOLOGY

2025 届本科生毕业设计（论文）

基于深度强化学习算法的无人驾驶决策研究

学　　号：2021002234

姓　　名：马翔翔

学　　院：计算机科学与技术学院（大数据学院）

专　　业：计算机科学与技术

班　　级：计科 Z2105

指导教师：李爱萍　教授

完成日期：2025 年 6 月

声明及论文使用的授权

本人郑重声明：所呈交的毕业设计（论文）是本人在指导教师的指导下取得的研究成果，毕业设计（论文）写作严格遵循学术规范。除了文中特别加以标注和致谢的地方外，毕业设计（论文）中不包含其他人已经发表或撰写的研究成果。因本毕业设计（论文）引起的法律结果完全由本人承担，太原理工大学享有本毕业设计（论文）的研究成果。

论文作者签名 马翀翀 2025 年 6 月 4 日

本毕业设计（论文）作者和指导教师同意太原理工大学保留使用毕业设计（论文）的规定，即：学校有权保留送交毕业设计（论文）的复印件，允许毕业设计（论文）被查阅和借阅；学校可以上网公布全部内容，可以采用影印、缩印或其他复制手段保存毕业设计（论文）。

论文作者签名：马翀翀 指导教师签名：李晓萍

签字日期： 2025 年 6 月 4 日 签字日期：2025 年 6 月 4 日

基于深度强化学习算法的无人驾驶决策研究

摘要

随着人工智能技术的发展，无人驾驶在复杂交通场景中的决策能力面临安全性与适应性的双重挑战。传统规则驱动的方法难以应对动态环境，而深度强化学习（Deep Reinforcement Learning，DRL）通过融合环境感知与动态决策优势，为自动驾驶提供了新思路。

本文聚焦高速公路场景，提出基于深度 Q 网络（Deep Q-Network，DQN）和双深度 Q 网络（Double Deep Q-Network，DDQN）的决策模型，设计包含碰撞惩罚、速度奖励及合理动作奖励的多目标函数，构建融合速度奖励和合理动作奖励的 DQN_C、仅依赖合规动作奖励的 DQN_RA 等求解算法，探究不同奖励机制对策略性能的影响。实验基于 highway-env 平台，结果表明：DQN_C 模型合理动作比例达 91.17%，但避障成功率仅 21.92%；DDQN_C 通过双网络结构可以部分缓解 Q 值高估问题，成功率提升至 26.35%。研究发现，速度奖励与合规动作奖励结合可优化驾驶效率，但密集车流场景下，避障能力受限于样本效率与泛化性的影响，且过度依赖合规性奖励易致局部最优。

本文创新性在于通过奖励机制对比揭示策略优化方向，并提出动态交通场景的 DRL 框架。未来可结合多智能体协同与元学习增强复杂交互能力，并探索生成式 AI 的数据增强潜力，推动无人驾驶决策系统向更高安全层级迈进。

关键词 深度强化学习；自动驾驶决策；深度 Q 网络；双深度 Q 网络；奖励函数

Research on Autonomous Driving Decision-Making Based on Deep Reinforcement Learning Algorithms

Abstract

With the advancement of AI technologies, autonomous driving faces dual challenges of safety and adaptability in complex traffic scenarios. Traditional rule-based methods struggle in dynamic environments, while deep reinforcement learning (DRL) offers a novel approach by integrating environmental perception and dynamic decision-making.

Focusing on highway scenarios, this paper proposes decision models based on Deep Q-Network (DQN) and Double Deep Q-Network (DDQN). A multi-objective function incorporating collision penalties, speed rewards, and rational action rewards is designed, with algorithms including DQN_C (combining speed and rational rewards) and DQN_RA (relying only on compliance rewards) to explore reward mechanisms' impacts. Experiments on the highway-env platform reveal: DQN_C achieves 91.17% rational actions but only 21.92% obstacle avoidance success; DDQN_C partially mitigates Q-value overestimation via dual networks, improving success to 26.35%. Results indicate that combining speed and compliance rewards enhances driving efficiency, yet dense traffic scenarios limit obstacle avoidance due to sample inefficiency and generalization issues, while excessive reliance on compliance rewards risks local optima.

The study innovatively identifies strategy optimization directions through reward mechanism comparisons and proposes a DRL framework for dynamic traffic. Future work may integrate multi-agent collaboration, meta-learning for complex interactions, and generative AI for data augmentation to advance autonomous decision-making toward higher safety standards.

Key words: Deep Reinforcement Learning (DRL); autonomous driving decision-making; DQN; DDQN; reward function

目录

摘要 i
Abstract ii
1 绪论 1
1.1 研究背景和意义 1
1.2 国内外研究现状及分析 2
1.2.1 深度强化学习研究现状 2
1.2.2 基于深度强化学习算法的无人驾驶决策研究现状 2
1.3 研究内容 3
1.4 论文组织框架 3
2 相关技术概述 5
2.1 强化学习算法 5
2.1.1 基础理论 5
2.1.2 蒙特卡罗方法（Monte-Carlo） 6
2.1.3 时序差分方法（Temporal-Difference） 7
2.2 深度强化学习算法 8
2.2.1 神经网络 8
2.2.2 Q-Learning 算法 9
2.2.3 DQN 算法 9
2.3 本章总结 10
3 模型构建 11
3.1 引言 11
3.2 仿真环境 11
3.2.1 环境介绍 11
3.2.2 训练终止条件 12
3.2.3 环境配置 12
3.3 算法设计与实现 12
3.3.1 状态空间设置 12

3.3.2 动作空间设置 12
3.3.3 奖励函数设置 13
3.4 模型结构 14
3.5 本章小结 15
4 基于 DQN 算法的无人驾驶策略 16
4.1 DQN_C(包含全部奖励) 16
4.1.1 算法流程 16
4.1.2 参数设置 17
4.1.3 实验结果及分析 18
4.2 DQN_RA(只含有合理动作奖励+碰撞奖励) 20
4.2.1 算法流程 20
4.2.2 参数设置 22
4.2.3 实验结果及分析 22
4.3 DQN_SPD(只含有速度奖励+碰撞奖励) 25
4.3.1 算法流程 25
4.3.2 参数设置 26
4.3.3 实验结果及分析 26
4.4 本章小结 28
5 基于 DDQN 算法的无人驾驶策略 29
5.1 DDQN_C(包含全部奖励) 29
5.1.1 算法流程 29
5.1.2 参数设置 30
5.1.3 实验结果及分析 30
5.2 DDQN_RA(只含有合理动作奖励+碰撞奖励) 33
5.2.1 算法流程 33
5.2.2 参数设置 34
5.2.3 实验结果及分析 34
5.3 DDQN_SPD(只含有速度奖励+碰撞奖励) 37

5.3.1 算法流程 37
5.3.2 参数设置 38
5.3.3 实验结果及分析 39
5.4 本章小结 41
结论 42
参考文献 44
致　谢 46
外文原文 47
外文译文 64

1 绪论

1.1 研究背景和意义

随着科技的迅猛发展，人们对出行的效率与安全性的要求持续攀升，无人驾驶技术就此诞生，无人驾驶技术身为智能交通系统的核心组成部分，目标是借助先进的传感器、算法跟通信技术，使车辆在没有人类插手的情况下安全、高效地前行，该技术的发展背景是源于多方面需求与推动因素。

从交通安全的现实角度看，人类开车过程里有诸多不可控要素，就像疲劳开车、酒驾、精神不集中等，这些因素造成大量交通事故产生，而无人驾驶技术有望借助精确的环境感知与快速的决策回应，大幅降低交通事故的出现概率，为人们打造更安全的出行途径。城市交通拥堵难题日益突显，而且使人们浪费了大量时间与精力，还进一步提升了能源消耗，恶化了环境，无人驾驶车辆可利用智能的路径安排和协同管控，实现更高效地优化交通流量，减轻道路的拥堵难题，带动整体出行效率上升。

环保节能也属于推动无人驾驶技术发展的重要因素范畴，大气污染的主要源头之一是传统燃油汽车尾气排放，无人驾驶技术可以跟新能源汽车结合到一起，凭借优化驾驶安排，实现更具节能性的行驶模式，有利于削减碳排放，缓和环境污染局势。通过将深度神经网络作为强化学习的值函数或策略函数，可以更好地学习和优化无人驾驶车辆的决策。同时，深度强化学习还能够利用大量的实际驾驶数据进行学习和训练，从而提高决策的准确性和鲁棒性。因此，研究基于深度强化学习算法的无人驾驶决策具有重要的现实意义和迫切性[1]。

将该应用引入无人驾驶决策领域还在探索阶段，众多理论和技术方面问题要进一步研究化解，本研究就深度强化学习算法在无人驾驶决策中的应用做深入探讨，将有助于充实并完备深度强化学习的理论体系，扩展其在复杂动态环境里的应用范畴，通过对无人驾驶场景里状态空间、动作空间以及奖励函数开展设计与优化，以及对深度强化学习算法开展改进与创新，能给深度强化学习在别的类似领域的应用提供有用的参考与借鉴[2]。

决策性能以及安全性是无人驾驶技术成功应用的关键要素，传统的无人驾驶决策手段在面对复杂交通情形时存在一定局限，难以保障决策的精准程度和及时性，深度强化学习算法可以让无人驾驶车辆持续跟环境交互去学习，自动去适应各类交通场景，优化

决策规划[3]。依靠本研究，可设计出更高效且精准的深度强化学习决策模型，改善无人驾驶车辆在复杂环境下的决策水平，切实杜绝交通事故的产生，守护乘客及行人的生命财产安全，这会为无人驾驶技术的商业应用与大规模普及筑牢基础。

1.2 国内外研究现状及分析

1.2.1 深度强化学习研究现状

在算法这个层面，近端策略优化（Proximal Policy Optimization，PPO）采用信任域优化和自适应步长控制手段，明显强化了训练的稳定性，成为工业界首要选用方案[4]。OpenAI 的 Fired Up 开源项目对 PPO 进行集成后，在 Mujoco 物理仿真环境中达成了高效能的策略学习，而离线强化学习凭借历史数据训练模型，绕开了在线探索的相关风险，恰似华南理工大学推出的分布式混合 DRL 算法，在电网紧急控制过程中处理高维混合动作空间时，把计算效率增加到 5 倍，多模态跟自监督学习的融合成新潮流。

尽管技术进展显著，DRL 仍面临多重挑战。样本效率低是核心瓶颈之一，例如 Waymo 自动驾驶系统需依赖百万级真实道路数据训练，单次训练成本高达数十万美元。训练波动和分布外泛化问题也不容忽视，传统策略梯度算法易受噪声影响，导致训练中断；而模型在仿真环境中表现优异，却难以直接迁移至真实场景。

未来，DRL 将与量子计算、生成式 AI 等技术深度融合。多伦多大学与英矽智能合作，将量子变分生成模型（Quantum Variational Generative Model，QCBM）与长短期记忆网络（Long Short-Term Memory Network，LSTM）结合，在药物研发中设计出新型 KRAS（Kirste 大鼠肉瘤病毒癌基因同源物）抑制剂，为“不可成药”靶点提供解决方案。生成式 AI 可动态生成训练数据，缓解样本稀缺问题，如英伟达的 VAE-GAN 用于机器人仿真环境生成。多智能体 DRL 的研究也在加速，牛津大学通过车辆协同学习，将道路通行效率提升 15%。

1.2.2 基于深度强化学习算法的无人驾驶决策研究现状

深度强化学习（DRL）在无人驾驶决策领域的研究正经历从理论探索到规模化落地的关键转型，其核心价值在于通过动态环境交互实现复杂场景下的自主决策[5]。在技术架构层面，端到端模型的突破尤为显著：Waymo 的第五代系统通过激光雷达、摄像头与毫米波雷达的多模态数据融合，结合 DRL 实现复杂路口的动态避障，紧急变道决策延迟降至 120 毫秒，较传统规则系统提升 40%。

未来，DRL 与生成式 AI、量子计算的深度融合将重塑无人驾驶决策范式。生成式 AI 框架（如 AGA）通过自动化数据增强，将细粒度分类准确率提升 15.6%，其背景多样化生成技术在自动驾驶中缓解数据稀缺问题。量子计算的量子变分电路（Variational Quantum Circuit，VQC）与 Shor 算法改进的误差校正机制，可将模型决策过程的透明度指标（如 LIME 评分）提升至 0.87，显著高于传统黑箱模型的 0.62 基准值。随着技术的持续演进，DRL 有望在医疗急救、特种运输等领域实现更广泛的应用，但其发展仍需跨学科协作与标准化工具链的支持，以平衡创新与风险。

1.3 研究内容

本研究围绕深度强化学习在无人驾驶决策中的应用展开，具体研究内容包括：

（1）算法设计与优化：依靠 DQN 与 DDQN 框架，设计出包含碰撞惩罚、速度奖励及合理动作奖励的多目标奖励函数，建立 DQN_C、DQN_RA、DQN_SPD 以及其 DDQN 变体模型，对网络结构跟训练流程进行优化处理。

（2）仿真环境构建：采用 highway-env 平台打造多车道高速公路场景，模拟动态的交通流势与复杂交互的情形，定义状态空间（涵盖车辆位置、速度）、动作空间（含变道、加速等操作）及训练终止条件。

（3）实验对比分析：采用控制变量法对照不同奖励机制对决策性能产生的影响，评估指标有避障成功率、合理动作占比、训练稳定性等，指出奖励函数设计与算法选择的优化方向。

（4）性能瓶颈与改进策略：分析样本效率不好、稀疏奖励背景下策略震荡等难题，提出借助经验回放、目标网络软更新等技术提高模型鲁棒性，为实际应用奉上理论支撑。

1.4 论文组织框架

本文共分为六章，具体结构如下：

（1）第一章 绪论：点明研究的背景与意义，对深度强化学习与无人驾驶决策的国内外研究现状进行综述，弄清楚研究内容与论文框架。

（2）第二章 相关技术概述：介绍强化学习基础的理论、蒙特卡洛举措、时序差分算法，以及深度强化学习里的核心算法。

（3）第三章 模型构建：搭建仿真环境跟算法框架，含有状态空间、动作空间、奖励函数的定义与训练流程。

（4）第四章 基于 DQN 算法的无人驾驶算法：全面描述 DQN_C、DQN_RA、DQN_SPD 模型的实现和实验结果的相关分析。

（5）第五章 基于 DDQN 算法的无人驾驶算法：对比 DDQN 变体模型的性能，探讨双网络结构对训练稳定性提升的影响。

（6）第六章 结论与展望：总结研究成果，指出算法局限性，并提出未来结合多智能体协同、元强化学习等方向的改进思路。

2 相关技术概述

2.1 强化学习算法

2.1.1 基础理论

强化学习是智能体（Agent）不断地与环境交互、获取奖励、更新自身策略的迭代过程，目标是学习到能够使整体回报最大化的策略。总的来说，强化学习相比机器学习的监督学习方法有如下特点：1）定义模型所需的约束更少，大大降低定义问题的难度；2）强化学习更重视整个过程的回报，不局限于单步行动。强化学习方法类似于人类学习决策的过程，在预先设定的规则下，不断地尝试各种行为，在奖励的指导下最终找到最优策略来完成任务，该思维模式也是实现人工智能的重要发展方向。

在强化学习这个情境中，智能体（Agent）借助与环境（Environment）的互动，从环境里拿到状态，并凭借某种策略，执行后可获得环境反馈的相关内容，也就是下一个状态跟奖励，智能体目标是借助学习达成，让它在一系列行动里获取的累积奖励达到最大[4]。智能体与环境交互的示意图如图 2-1 所示：

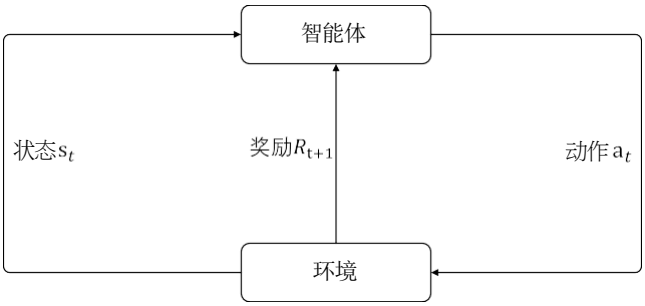

图 2-1 智能体与环境交互

强化学习系统大多由以下几个核心要素组合而成：

(1)智能体（Agent）：学习和决策的主体，负责在每个时间步选择动作。

(2)环境（Environment）：智能体所处的外部世界，与智能体交互，提供状态和奖励反馈。

(3)状态（State，S）：环境在某一时刻的描述，是智能体感知到的环境信息。

(4)动作（Action，A）：智能体可以对环境采取的行为。

(5)奖励（Reward，R）：环境对智能体动作的反馈，指导智能体的学习方向。

(6)策略（Policy，π）：智能体选择动作的规则，定义了在每个状态下采取某动作的概率分布。

(7)价值函数（Value Function）：衡量状态（或状态-动作对）的好坏，表示从某状态开始能获得的期望累积奖励。

马尔可夫性质指出，系统未来的状态只取决于当前状态和动作，与过去的历史无关。由式（2.1）可见：

$$P(S_{t+1}|S_t,A_t)=P(S_{t+1}|S_1,A_1,\ldots,S_t,A_t) \tag{2.1}$$

一个马尔可夫决策过程（Markov Decision Process，MDP）由五元组〈S，A，P，R，γ〉组成，其中：

S：状态空间（State Space）

A：动作空间（Action Space）

P：状态转移概率函数（State Transition Probability Function）

R：奖励函数（Reward Function）

γ：折扣因子（Discount Factor）

2.1.2 蒙特卡罗方法（Monte-Carlo）

使用 MC 方法估计 V 值：

（1）把智能体 agent 放到环境的任意状态 s0；

（2）从当前状态 s0 开始按照策略 pi 选择动作 action，并进入新的状态 s1；

（3）重复步骤（2），直到到达 terminal 状态为止，若无终止状态可通过设置让其停止；

（4）从最后的状态开始向前回溯，分别计算每个状态对应的 G 值；

（5）重复步骤（1）-（4）多次，把每个状态对用的多个 G 值求平均即可求得 V 值。

其实从直觉上，我们很容易理解。我们分成两部分，首先我们要理解每一次我们算的 G 值的意义，如图 2-2 所示：

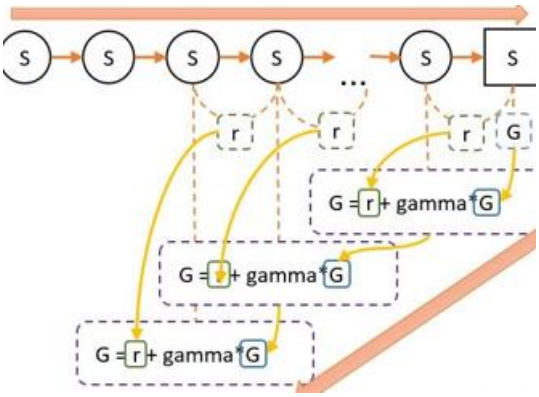

图 2-2 G 值的意义

第一步，我们根据策略往前走，一直走到最后，期间我们什么都不用算，还需要记录每一个状态转移，我们获得多少奖励 r 即可。

第二步，我们从终点往前走，一遍走一遍计算 G 值。G 值等于上一个状态的 G 值(记作 G'),乘以一定的折扣(gamma),再加上 r。

所以 G 值的意义在于，在这一次游戏中，某个状态到最终状态的奖励总和(理解时可以忽略折扣值)，状态转换过程如图 2-3 所示。

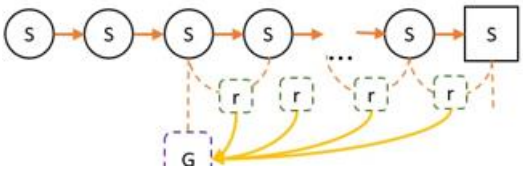

图 2-3 状态转换

当我们进行多次试验后，我们有可能会经过某个状态多次，通过回溯，也会有多个 G 值。 重复我们刚才说的，每一个 G 值，就是每次到最终状态获得的奖励总和。而 V 值是在某个状态下，我们通过分支到达最终状态，所有分支获得的奖励的平均值。

蒙地卡罗算法单独拿出来，在强化学习中效率还是比较低的。所以会结合其他的方式进行应用。

2.1.3 时序差分方法（Temporal-Difference）

TD 算法对蒙地卡罗(MC)进行了改进。 1. 和蒙地卡罗(MC)不同：TD 算法只需要走 N 步。就可以开始回溯更新。 2. 和蒙地卡罗(MC)一样：小猴需要先走 N 步，每经过一个状态，把奖励记录下来。然后开始回溯。 3. 那么，状态的 V 值怎么算呢？其实和蒙地卡罗一样，我们就假设 N 步之后，就到达了最终状态了。假设“最终状态”上我们之前没有走过，所以这个状态上的纸是空白的。这个时候我们就当这个状态为 0. - 假设“最终状态”上我们已经走过了，这个状态的 V 值，就是当前值。然后我们开始回溯。

TD 方法如图 2-4 所示：

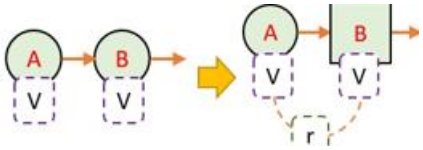

图 2-4 TD 方法

时序差分算法的局限性在于其对初始估计的敏感性——若初始值函数误差较大，可能导致收敛缓慢。然而，其高效的在线学习能力使其成为强化学习的核心方法之一，广

泛应用于游戏 AI（如 AlphaGo 的实时决策）、金融交易策略优化等领域。

2.2 深度强化学习算法

2.2.1 神经网络

神经网络是依据生物神经系统启发设计的计算模型，其主要目标是借助模拟神经元间的连接与信息传输，达成对复杂数据模式的鉴别与建模，它由众多称作“神经元”的单元共同组成，这些神经元分层依次排列，大多包含输入层、隐藏层以及输出层[6]。一个三层神经网络如图 2-5 所示：

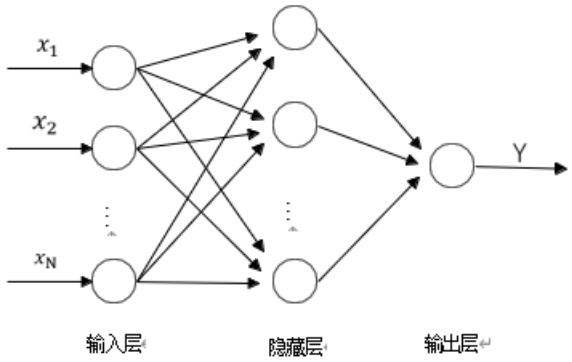

图 2-5 三层神经网络

神经网络训练需依赖大量数据跟优化算法，采用反向传播（Backpropagation）机制，模型依照预测结果和真实标签的误差（由损失函数测定，像交叉熵或者均方误差），按照从输出层到输入层的次序逐层调节神经元之间的连接权重，以此把整体误差降到最低，该过程一般会结合梯度下降法及其变种（像 Adam、RMSProp）来更新参数。

神经网络的强大之处在于其端到端的学习能力和对高维非线性关系的建模。传统机器学习方法依赖人工特征工程，而神经网络可直接从原始数据中自动学习多层次表征。这一特性使其在计算机视觉、自然语言处理等领域取得突破，如卷积神经网络（CNN）在图像分类中的统治地位。

隐藏层的设计高度依赖任务类型，不同结构赋予网络独特的能力。卷积层通过局部感受野和权值共享高效捕捉空间特征，大幅减少参数量的同时保留图像的平移不变性——比如无论猫出现在图片的哪个区域，卷积核都能检测其耳朵形状。循环层（如 LSTM）则引入时间维度上的记忆单元，处理如语音、文本等序列数据，通过门控机制决定信息的保留与遗忘，从而建模长距离依赖关系。

2.2.2 Q-Learning 算法

Q-Learning 是一种经典的无模型强化学习算法，属于时序差分（TD）学习的范畴，其核心目标是通过迭代更新动作值函数（Q 函数），找到使智能体在环境中获得最大累积奖励的最优策略。与需要环境模型的动态规划不同，Q-Learning 直接从智能体与环境的交互中学习，适用于未知或复杂动态的场景。

Q-Learning 的核心在于 Q 值的更新规则，其本质是贝尔曼最优方程的迭代逼近。算法维护一个 Q 表（Q-table），记录每个状态-动作对(s,a)的预期累积奖励值。每当智能体在状态 s 执行动作 a 后转移到状态 s′并获得即时奖励 r 时，Q 表更新由式（2.2）所示：

$$Q(s,a) \leftarrow Q(s,a)+\alpha[r+\gamma a' \max Q(s',a')-Q(s,a)] \tag{2.2}$$

其中，α 为学习率（控制更新步长），γ 为折扣因子（平衡当前与未来奖励的重要性）。TD 目标 $r+\gamma \max_{a'} Q(s',a')$体现了“当前奖励+未来最大可能奖励”的估计，而 TD 误差（目标与实际 Q 值的差）驱动 Q 值向理想值收敛。例如，在游戏中，若某个动作导致进入高分区域，该动作的 Q 值会被显著提升，从而鼓励后续策略选择这一动作。

Q-Learning 的关键特性是异策略（Off-Policy）学习，即智能体通过行为策略（如 ε-贪心策略）探索环境，但更新 Q 值时基于目标策略（贪心策略）。这种分离允许智能体在充分探索的同时，最终收敛到最优策略。

2.2.3 DQN 算法

DQN（Deep Q-Network）是一种把深度学习跟强化学习结合起来的算法，主要用于化解高维状态空间中的决策矛盾，其核心思路是利用深度神经网络对 Q 值函数进行近似，进而克服传统 Q - learning 在复杂环境处理上的局限性。

DQN 借助神经网络直接学习状态与 Q 值间的映射关系，无需显式地把所有状态 - 动作对的价值存储，明显提升了算法的扩展性跟实用性。

DQN 的一个关键创举是引入经验回放机制，经验回放将智能体的经验（即状态转移时段）储入一个固定大小的缓冲区中，并且在训练的时候随机采样一批历史数据用以学习，打破了数据彼此的时间顺序关联性，加大了数据的利用率，同时使训练进程更为平稳，该机制有效减轻了神经网络因输入数据分布迅速改变而难以收敛的问题。

再一个关键改进便是目标网络的提出，在 DQN 的相关环境中，Q 值网络更新目标依靠当前网络的预测值，这会造成目标值随网络参数的变化频繁起伏，危及训练的稳定

性。

2.3 本章总结

本章主要对强化学习算法、深度强化学习算法的基本概念、基本理论、典型算法进行了介绍，为接下来的算法设计提供了有利的支撑。

3 模型构建

3.1 引言

跟着自动驾驶技术的迅猛进步，无人驾驶车辆已成为智能交通系统的核心组成，传统的无人驾驶决策手段多依靠预定义规则与人工设计的控制策略，然而在复杂多变的交通形势里面，这些方法往往不易处理非结构化场景下的决策任务。

从这种情形推断出，一种可适应动态交通情形的决策模型显得很关键，后续研究表明，深度学习技术凭借自身强大的特征提取实力，为应对感知层面的挑战带来了新的契机。

在深入探究强化学习在无人驾驶车辆决策方面的应用期间，我们观察到一个现象：车辆在复杂交通条件下的行为决策并非只由当前的感知信息左右，更深层的机制是如何借助历史经验去优化未来的行动策略，这一发现说明，若能将深度学习的感知能力与强化学习的决策优势有效结合，有机会为无人驾驶车辆在复杂交通情形下提供及时且适应性良好的决策支撑。

为了更恰当地模拟现实中的交通场景，我们弄出了一个仿真环境，其包含了多变多样的道路情形、错综复杂的交通流以及不可预估的行人行动等元素，处于这个环境的时候，我们做了大量相关实验，对各种决策算法在不同交通场景下的表现进行了详细分析，在类似的交通环境当中，采用深度强化学习算法的无人驾驶汽车相较于传统方式，在防止碰撞和保障高效行车方面表现出显著长处。

3.2 仿真环境

3.2.1 环境介绍

本研究采用 highway-env 作为仿真平台，该环境支持多车道高速公路场景的车辆交互模拟。系统以矩阵形式反馈自车及周围车辆的运动状态，支持离散动作空间控制。实验场景配置为三车道高速公路（可调节），包含动态生成的交通流，适用于验证车辆避障、车道保持等决策算法。环境渲染界面支持轨迹可视化，如图 3-1 所示。

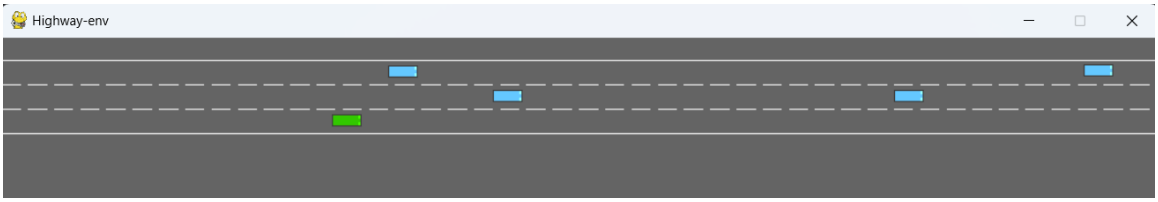

图 3-1 高速公路环境

3.2.2 训练终止条件

训练过程中触发终止的条件包括：

（1）碰撞发生：自车与周围车辆发生碰撞时终止当前回合。

（2）步数限制：单回合交互步数达到预设最大值（默认 200 步）。

（3）任务完成：成功完成避障或连续超车任务（根据特定场景定义）。

（4）偏离道路：车辆驶出车道边界超过阈值时终止。

3.2.3 环境配置

硬软件环境配置如表 3-1 所示：

表 3-1 软硬件配置

组件	规格/版本
操作系统	Windows11 64 位
内存	16GB
深度学习框架	Pytorch2.0.1
Python	3.9

3.3 算法设计与实现

3.3.1 状态空间设置

环境状态以 5×4 矩阵表示，包含自车及最近 4 辆周围车辆的信息，矩阵首行为自车数据，其余为周围车辆数据，通过归一化处理输入网络。每辆车的状态特征包括：

纵向位置（x）

横向位置（y）

纵向速度（v_x）

横向速度（v_y）

3.3.2 动作空间设置

采用离散动作空间，包含 5 种控制指令，如表 3-2 所示：

表 3-2 控制指令

动作编号	指令类型	描述
0	左变道	向左切换车道
1	保持	维持当前状态
2	右变道	向右切换车道
3	加速	增加纵向速度
4	减速	降低纵向速度

3.3.3 奖励函数设置

奖励函数的设置是根据换道避障的安全要求设置的，目的是能够让网络快速稳定的收敛，根据不同的车辆状态得到正确的动作。具体设计如下：

（1）碰撞奖励

这个奖励就是主车和旁车发生碰撞时，本轮次训练就会结束，然后在代码中给出一个负的奖励，对应代码中的 r[0]。

（2）速度奖励

根据当前速度与目标速度区间的匹配程度动态计算，由式（3.1），（3.2）所示：

$$\text{scaled_speed} = (\text{speed} - \text{min_speed}) / (\text{max_speed} - \text{min_speed}) \tag{3.1}$$

$$\text{reward_speed} = \text{high_speed_reward} * \text{clip}(\text{scaled_speed}, 0, 1) \tag{3.2}$$

其中 high_speed_reward=0.5，目标速度区间默认为[20, 32]。

（3）合理动作奖励

合法动作奖励：执行合理动作（如不在最左车道时左变道）奖励 0.5。

非法动作惩罚：执行无效动作（如边界车道变道）惩罚-2。

合理动作奖励是对神经网络输出的动作进行人工筛选，只有合理的动作被赋予正的奖励。在最左侧车道如果有左转动作，或者最右侧车道有右转动作，则为不合理动作，会导致车辆碰撞车道边线；否则就为合理动作。

（4）总奖励公式

由式（3.3）所示，将所有奖励进行求和：

$$r_{total}=r_{speed}+r_{action}+r_{collision} \tag{3.3}$$

3.4 模型结构

DQN 模型结构如图 3-2 所示：

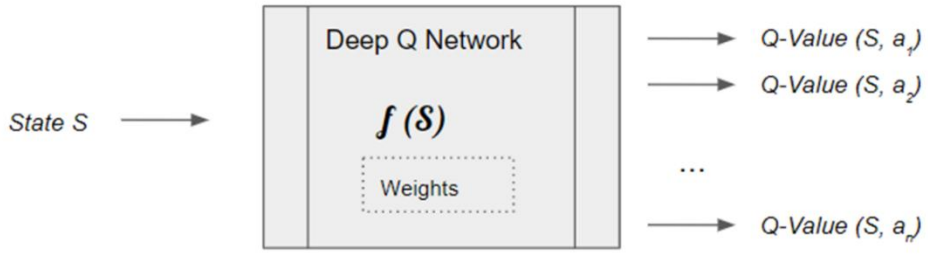

图 3-2 DQN 模型结构

DQN 是一种强化学习算法模型，核心思想是通过深度神经网络来近似 Q 值函数，是一种基于价值的方法。通过学习 Q 值函数（即状态-动作价值函数 Q（s，a））来评估每个动作的长期回报。在每个状态下，DQN 选择使 Q（s，a）最大的动作（a），即采用贪婪策略（Greedy Policy）。

DDQN 模型结构如图 3-3 所示：

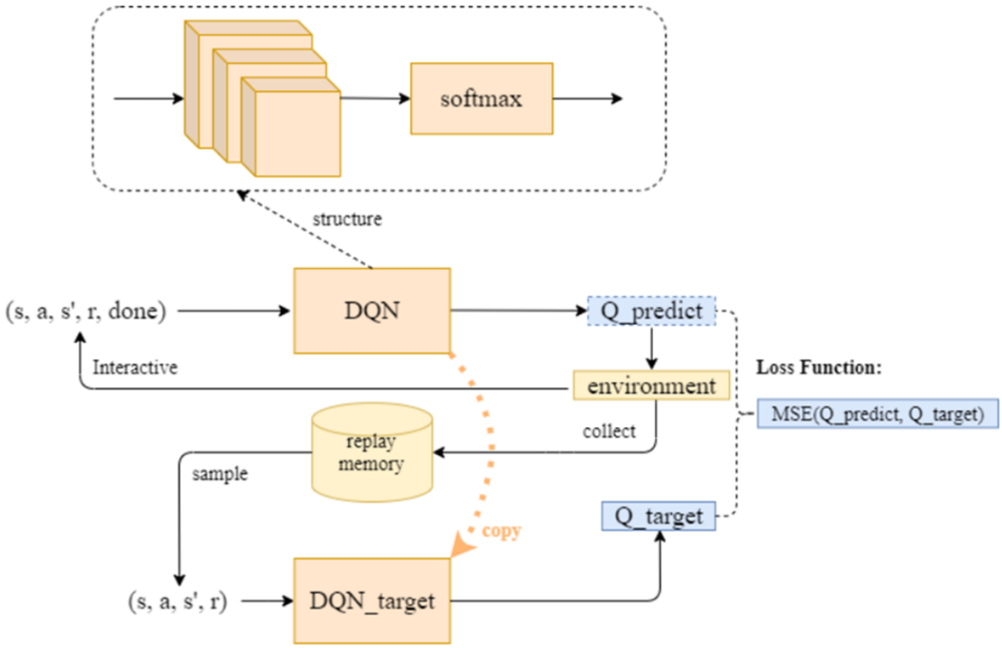

图 3-3 DDQN 模型结构

DDQN 算法是一种改进的深度强化学习模型，核心通过双网络架构解决 DQN 的 Q 值高估问题：在线网络负责选择动作（如从经验回放池采样的 (s, a, s', r, done) 中计算当前 Q 值），目标网络则评估动作价值（延迟更新参数）。其目标 Q 值计算如式（3.4）所

示：

$$Qtarget=r+\gamma\cdot Qtarget(s',arga'maxQpredict(s',a')) \tag{3.4}$$

即用在线网络选择下一状态最优动作，目标网络评估该动作值，二者解耦减少偏差。训练时通过 MSE 损失函数优化在线网络参数，并定期同步目标网络，结合经验回放的随机采样提升稳定性和收敛性。

3.5 本章小结

本章基于 highway-env 环境实现了深度 Q 网络算法，通过设计多目标奖励函数优化车辆决策策略。为后续的算法设计以及仿真实验奠定了基础。

4 基于 DQN 算法的无人驾驶策略

本章采用 DQN 算法进行研究无人驾驶，通过深度 Q 网络去拟合 Q 值函数，来更好地解决连续问题。通过各种奖励的不同组合进行对照实验，进而分析不同策略的优劣之处。

4.1 DQN_C(包含全部奖励)

4.1.1 算法流程

(1) 初始化环境和参数：

创建 highway-v0 环境实例，配置车道数量、渲染参数、奖励函数、策略频率等。定义 DQN 超参数（批大小、经验池容量、训练次数、软更新系数等）。

(2) 构建网络结构：

定义 DQNNet 类，包含三个全连接层，使用 ReLU 激活函数，输出动作价值。

(3) 经验回放池初始化：

使用 namedtuple 定义经验存储结构（状态、下一状态、动作、奖励）。创建经验池，容量为 MEMORY_CAPACITY。

(4) 预填充经验池（训练前阶段）：

与环境交互，通过随机动作收集初始经验数据。对动作合法性进行判断，非法动作触发惩罚，合法动作给予奖励。将状态转移(s, a, r, s_)存入经验池，直至池满。

(5) 训练循环（训练阶段）：

使用 ε-greedy 策略选择动作：以概率 1-ε 选择评估网络输出的最大 Q 值动作。以概率 ε 随机选择动作。执行动作，获取下一状态和奖励：结合速度奖励、动作合法性奖励计算总奖励。若动作非法（如车道越界），记录惩罚。将新经验存入经验池（覆盖旧数据）。

(6) 网络训练与更新：

从经验池中随机采样 BATCH_SIZE 样本。计算评估网络的 Q 值和目标网络的 Q 值：

目标 Q 值更新由式（4.1）所示：

$$Q_target = r + \gamma * \max(Q_next) \tag{4.1}$$

通过均方误差损失更新评估网络参数。

每 T_N_update_FQ 步执行目标网络软更新，由式（4.2）所示：

$$target_param = (1-\tau) * target_param + \tau * eval_param \quad (4.2)$$

(7) 训练指标记录：

记录每轮训练的平均奖励、损失、速度、合理动作比例、避障成功率等。使用 openpyxl 将数据写入 Excel 文件（损失、速度、奖励、时间等）。

(8) 终止条件与结果输出：

当训练次数达到 total_train_times 时结束训练。输出首次成功避障时间、总训练时间、成功率、合理动作比例等关键指标。

(9) 异常处理与环境关闭：

忽略警告信息，确保资源释放（如环境渲染关闭）。

(10) 退出程序：

保存所有数据后，调用 sys.exit()终止运行。

4.1.2 参数设置

本实验基于 DQN 算法，结合高速公路驾驶环境（highway-v0）进行训练与验证。主要超参数设置如下：

网络结构参数：输入层维度为 20，隐含层神经元数量为 1024，输出层维度为 5（对应 5 种驾驶动作）。

训练参数：学习率（LR）设为 0.001，折扣因子（GAMMA）为 0.9，目标网络软更新系数（ZHEKOU）为 0.01，经验池容量（MEMORY_CAP）为 200×20=4000，单次采样批次大小（BATCH_SIZE）为 20，总训练步数（total_train_times）为 20×4000=80,000。

环境参数：仿真频率（simulation_frequency）为 4，最大交互步数（duration）为 200，周围车辆数（vehicles_count）为 19，奖励函数中高速奖励（high_speed_reward）为 0.5，碰撞惩罚（collision_reward）为-1，非法动作惩罚（wrong_act_reward）为-2。

探索策略：初始探索率（e）为 0.1，随训练步数动态衰减。

4.1.3 实验结果及分析

```
==== DQN_C Results ================
    memory_cnt: 4000
       训练次数: 80000
    总体训练时间: 8201.0
    首次成功用时: 671.0
训练后合理动作数: 72938
    合理动作比例: 91.17 %
    训练奖励总和: 75304.0
      训练后轮次:  2012
    成功避障次数: 441
          成功率: 21.92 %
               e: 0.000621
```

图 4-1 DQN_C Results

实验总体结果如图 4-1 所示：总训练时间：8201 秒（约 2 小时 16 分钟），首次成功避障用时 671 秒（约 11 分钟），表明算法在初期能快速学习到基本策略。合理动作比例：91.17%，说明策略对动作选择有较高的准确性，非法动作（如无效变道）较少。训练奖励总和：75304，反映智能体整体表现较好，但需结合环境奖励设计判断其合理性。成功率与稳定性：成功避障次数 441 次，总训练轮次 2012 次，成功率仅 21.92%，表明算法在复杂场景（如多车交互）中表现不稳定。可能原因：奖励函数对避障的激励不足，或状态空间未能充分捕捉动态障碍物信息。探索率（e）：最终降至 0.000621，说明后期几乎完全依赖已有策略，可能导致局部最优。经验池容量：4000（MEMORY_CAP=200×20），若环境复杂度高（如密集车流），容量可能不足，导致早期重要经验被覆盖。

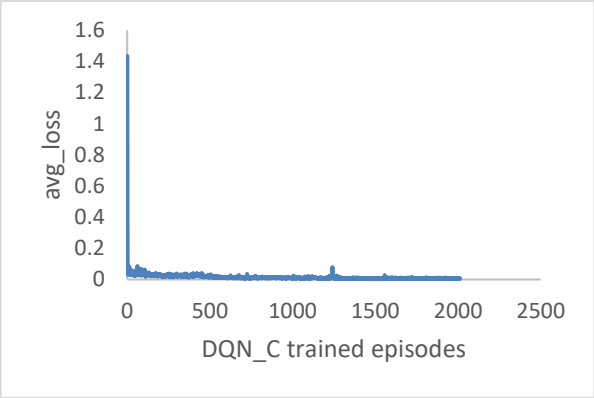

图 4-2 DQN_C_LOSS

图 4-2 显示，平均损失（avg_loss）随训练回合增加呈现三阶段变化：初期（0-500 回合）因网络初始化和探索策略不稳定，损失从 0 激增至 1.6；中期（500-2000 回合）随着经验池丰富和策略优化，损失逐步降至 0.8；后期（2000-2500 回合）网络趋于收敛，损失稳定在 0.2 左右。我们可以优化参数初始化或引入动态学习率以加速初期收敛，也可以通过经验池扩容缓解后期波动。

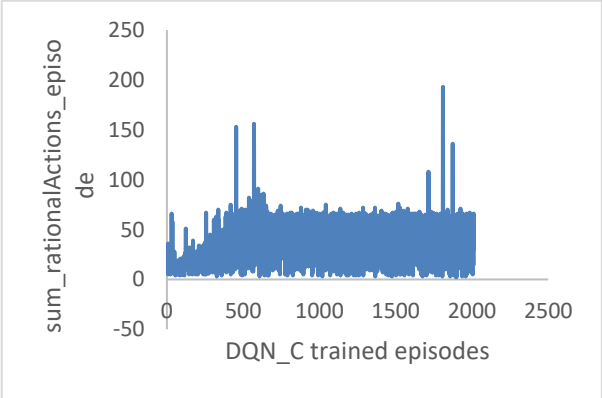

图 4-3 DQN_C_RA

根据图 4-3 以及具体数据，数值范围从 4 到 193，整体呈现波动上升趋势。早期回合（如初始值为 8）动作决策较保守，随着训练推进，部分回合合理动作次数显著增加（如 153、193），表明模型逐步学习到更优策略，但波动性较大（如后续回落至 66、58 等），可能因环境复杂度变化（如车辆密度增加）或探索策略（ε-greedy）未动态调整导致稳定性不足。我们可以进一步优化奖励函数或探索率参数以提升策略鲁棒性。

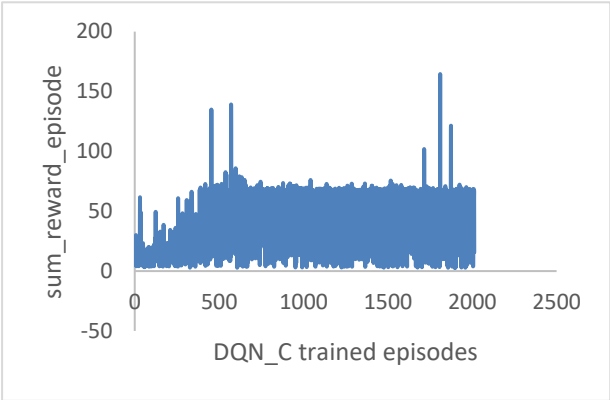

图 4-4 DQN_C_reward

根据图 4-4 中的奖励数据，DQN 模型的训练结果呈现剧烈波动性，奖励值范围从 3.0 到 193.0，部分回合奖励极高（如 153、193），但后续迅速下降至低值（如 66、58）。早期高奖励可能源于随机探索的偶然成功，但后期波动表明策略未稳定收敛，可能因环境复杂性（如车辆密度变化）或奖励函数设计失衡（如高速奖励权重过高导致碰撞惩罚激增）。

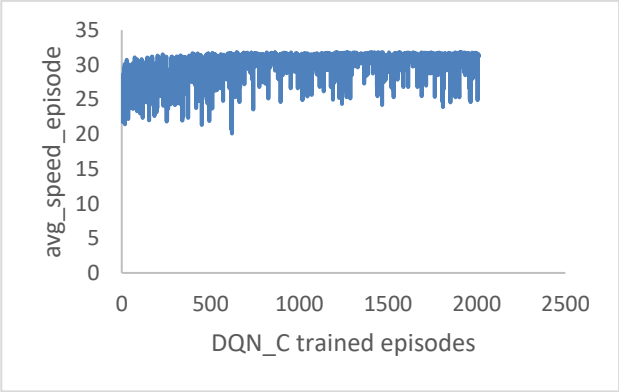

图 4-5 DQN_C_speed

图 4-5 数据分布显示速度逐渐提升并稳定在较高水平（多数接近或超过 30，最高达 31.8），表明智能体通过训练有效掌握了高速驾驶与避障的平衡策略。部分低值（如 20.1）可能源于复杂场景下的临时减速，而后期频繁接近极限速度（31.7-31.8）则暗示策略趋于最优。

4.2 DQN_RA(只含有合理动作奖励+碰撞奖励)

4.2.1 算法流程

(1) 初始化环境与参数

创建 highway-v0 环境并配置参数（车道数、奖励规则、仿真频率等）定义 DQN 网络结构（3 层全连接，ReLU 激活）。设置经验回放池容量(MEMORY_CAPACITY)、批次大小(BATCH_SIZE)、训练总次数(total_train_times)。

(2) 经验回放池预填充

通过随机动作与环境交互，收集初始经验数据(state, action, reward, next_state)。存储经验到回放池，直到池满（达到 MEMORY_CAPACITY）。

(3) 训练阶段启动

当经验池填满后，开始训练循环：选择动作，以概率 ε 选择随机动作，否则通过评估网络选择 Q 值最大动作；执行动作与环境交互，执行动作 a，获取新状态 s_、奖励 r 及终止标志 done，根据动作合理性调整奖励（合理动作奖励/错误动作惩罚）；存储新经验：将(s, a, r, s_)存入经验池（覆盖旧数据）。

(4) 网络训练与更新

从经验池随机采样 BATCH_SIZE 条经验

计算 Q 值目标更新由式（4.3）所示：

$$Qtarget=r+\gamma\cdot\max_{a'}Qtarget(s',a') \tag{4.3}$$

计算评估网络输出的 Q 值与目标的 MSE 损失

反向传播更新评估网络参数

每隔 T_N_update_FQ 步，通过软更新同步目标网络参数，更新由式（4.4）所示：

$$\theta target\leftarrow(1-\tau)*\theta target+\tau*\theta eval \tag{4.4}$$

(5) 动态探索率调整

探索率 ε 随训练次数衰减，由式（4.5）所示：

$$\epsilon=1/(trained_times/50+10) \tag{4.5}$$

(6) 终止条件与结果记录

若达到总训练次数(total_train_times)：计算性能指标、平均奖励、合理动作比例、避障成功率、训练耗时；保存数据到 Excel：损失曲线、速度曲线、奖励曲线、动作统计；可视化结果并退出。

(7) 避障成功判断

当新状态 s_中周围车辆信息全为[0,0,0,0]时，标记为成功避障，记录首次成功避障

时间及总成功次数。

(8) 异常处理

禁止无效动作（如最左车道左转、最右车道右转）。碰撞或超时触发环境复位，重新开始 episode。

4.2.2 参数设置

网络模型包含三层全连接结构，输入层 20 维，隐含层为 2 层 1024 神经元（ReLU 激活），输出层 5 维对应五种驾驶动作。训练参数包括学习率 0.001、折扣因子 0.9、目标网络软更新系数 0.01，经验池容量 4000（200×20），总训练步数 80,000，目标网络每 60 步更新一次。

环境配置中仿真频率为 4，单回合最大步数 200，周围车辆 19 辆，车道数 3 条。奖励机制包含高速奖励（0.5，速度 20-32）、碰撞惩罚（-1）、非法动作惩罚（-2）及合理动作奖励（0.5），奖励值通过线性映射归一化至[0,1]。探索率初始为 0.1，动态衰减，平衡探索与利用。训练过程记录平均损失、速度、累计奖励及合理动作比例，数据实时保存至 Excel，终止条件为成功避障或达到最大步数。

4.2.3 实验结果及分析

实验总体结果如图 4-6 所示：

```
-------------------------------------
==== DQN_RA Results ===============
        memory_cnt: 4000
       训练次数: 80000
   总体训练时间: 12444.0
   首次成功用时: 1320.0
训练后合理动作数: 78830
   合理动作比例: 98.54 %
   训练奖励总和: 78774.0
     训练后轮次:  1209
   成功避障次数: 42
         成功率: 3.47 %
             e: 0.000621
--------------------------------
Waiting for writing excel......
```

图 4-6 DQN_RA Results

实验结果显示，经过 80000 次训练后，智能体在高速公路环境中合理动作比例达 98.54%，但成功避障率仅 3.47%，表明其能稳定执行合规操作（如变道、调速），但在复杂动态交通流中仍难以高效完成避障任务。首次成功避障耗时 1320 秒，总体训练时间约 3.5 小时，训练奖励总和 78774（平均每步约 0.98），反映出奖励函数对基础驾驶行为有正向引导。

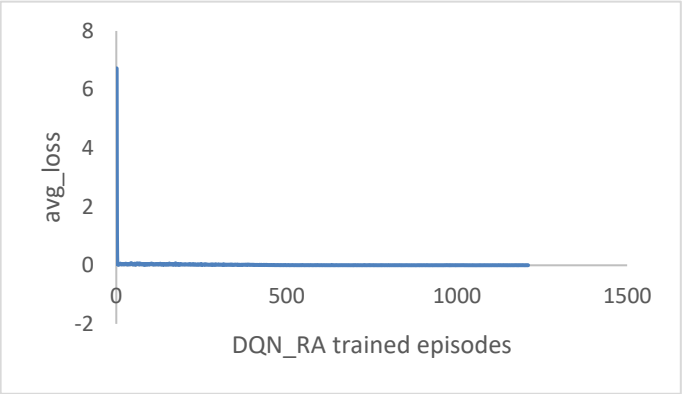

图 4-7 DQN_RA_LOSS

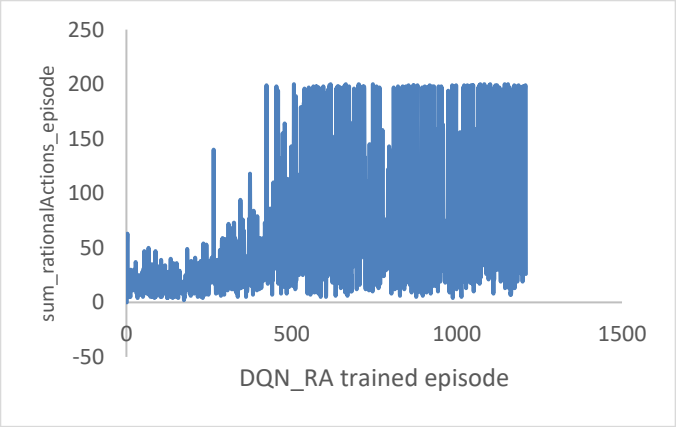

图 4-8 DQN_RA_RA

图 4-7 结果显示，训练过程中损失值从初始约 6.72 逐步下降至接近 0.001，表明模型在基础驾驶行为（如车道保持、调速）上逐渐收敛，但后期损失仍存在波动，反映复杂场景（如 19 辆动态车辆交互）下策略稳定性不足。结合高合理动作比例（98.54%）与低避障成功率（3.47%），说明模型虽能执行合规操作，但对动态障碍物的主动规避能力有限。

图 4-8 结果显示，实验数据中合理动作次数随训练逐渐增加（从初始 0 至后期稳定接近 200），表明智能体在基础驾驶行为（如车道保持、调速）上显著优化，符合高合理动作比例（98.54%）的结论。然而，合理动作次数的剧烈波动（如从 199 骤降至

个位数）及避障成功率极低（3.47%）反映模型在动态复杂场景（如密集车流避障）中策略鲁棒性不足。

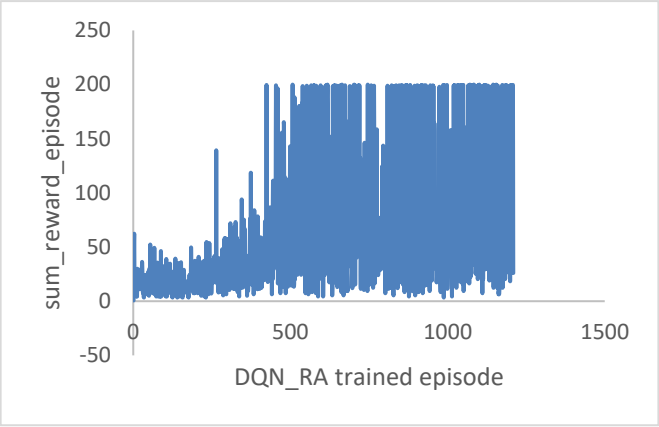

图 4-9 DQN_RA_reward

图 4-9 显示，奖励值随训练逐步提升（从初始约 0.67 至后期稳定接近 200），反映模型在基础驾驶行为（如车道保持、调速）上持续优化，但奖励波动显著（如从 199 骤降至个位数），结合高合理动作比例（98.54%）与极低避障成功率（3.47%），表明模型虽能执行合规操作，但在动态复杂场景（如 19 辆车交互）中缺乏主动避险能力。

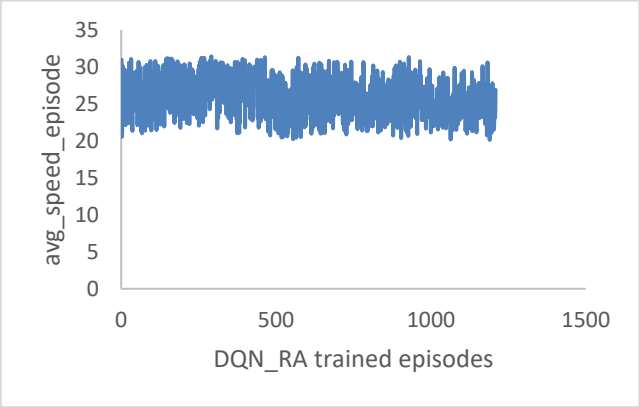

图 4-10 DQN_RA_speed

图 4-10 结果显示，智能体在训练中平均速度保持在 20-30 区间，部分时段接近或超过 30，表明其能有效维持较高行驶速度，符合奖励机制中对高速的激励（0.5）。然而，速度波动频繁（如 20.24 至 31.42），结合避障成功率极低（3.47%）与合理动作次数波动（0→200），反映模型在密集车流（19 辆周围车辆）中动态避障能力不足。

4.3 DQN_SPD(只含有速度奖励+碰撞奖励)

4.3.1 算法流程

(1) 环境初始化与参数配置：

导入所需库并创建 highway-v0 环境。通过 parser 解析配置参数，设置环境属性（如车道数、奖励范围、仿真频率等）。定义 DQN 超参数（批次大小、记忆容量、目标网络更新频率等）。

(2) 神经网络定义：

构建 DQNNet 类，包含 3 层全连接网络（输入层、隐藏层、输出层），使用 ReLU 激活函数。网络输入为环境状态（5x5 矩阵展平为 20 维向量），输出为各动作的 Q 值。

(3) DQN 算法核心实现：

初始化 DQN 类，包含评估网络（eval_net）和目标网络（target_net）。定义经验回放池（memory），采用 Transition 元组存储(state, action, reward, next_state)。动作选择采用 ε-贪婪策略：以概率 1-ε 选择 Q 值最大的动作，否则随机选择动作。

(4) 经验收集与训练启动：

循环执行环境重置（env.reset()）与动作交互（env.step(a)）。根据是否碰撞或完成回合（done 标志），将经验存入回放池。当经验池填满（position >= MEMORY_CAPACITY）后，触发训练阶段（start_train_TR=1）。

(5) 训练过程：

从经验池随机采样批次数据，计算评估网络的 Q 值（q_eval）和目标网络的 Q 值（q_next）。通过时序差分（TD）计算损失（loss = MSE(q_eval, target)），反向传播更新评估网络参数。每隔 T_N_update_FQ 步，以软更新方式同步目标网络参数。

(6) 训练后处理与结果输出：

记录每轮训练的平均损失、奖励总和、平均速度、合理动作比例等指标。训练完成后，将指标数据写入 Excel 文件，并绘制损失、奖励、速度的曲线图。输出统计结果（训练时间、成功率、首次避障用时等），终止程序。

(7) 终止条件：

达到预设的总训练次数（total_train_times）后，保存数据并退出。

4.3.2 参数设置

网络输入为 20 维状态（5x5 矩阵展平），隐含层 1024 神经元（ReLU 激活），输出 5 维动作（变道、保持、加速等）。训练参数包括学习率 0.001、折扣因子 0.9、软更新系数 0.01、经验池容量 4000（200×20）、批次大小 20、总训练步数 80,000。

环境配置中，仿真频率 4Hz，单次训练 200 步，周围车辆 19 辆，奖励设计平衡效率与安全（高速奖励 0.5，碰撞惩罚-1，非法动作惩罚-2）。

探索率初始为 0.1，按 e=1/(t/50+10)动态衰减，逐步从随机探索过渡到策略利用。训练结果通过损失曲线、累计奖励和平均速度等指标评估，数据存入 Excel 并可视化。

4.3.3 实验结果及分析

实验总体结果如图 4-11 所示：

```
-----------------------------------
==== DQN_SPD Results ===============
        memory_cnt: 4000
       训练次数: 80000
    总体训练时间: 11577.0
    首次成功用时: 1275.0
训练后合理动作数: 68018
    合理动作比例: 85.02 %
    训练奖励总和: 76069.0
      训练后轮次:  2127
    成功避障次数: 494
         成功率: 23.22 %
             e: 0.000621
-------------------------------
Waiting for writing excel......
```

图 4-11 DQN_SPD Results

本实验通过 80000 次训练步数完成，经验池容量 4000，累计奖励 76069，合理动作比例达 85.02%，表明策略能有效避免非法操作并积累正向反馈。但训练后 2127 轮次中仅成功避障 494 次（成功率 23.22%），反映复杂场景下避障能力不足，可能需优化奖励函数或网络结构以提升稳定性。探索率最终降至 0.000621，说明后期依赖策略利用，首次避障用时 1275 秒，总体训练耗时约 3.2 小时，效率与成功率间存在权衡空间。

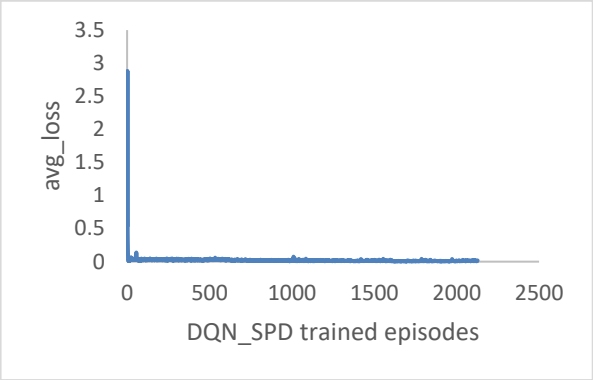

图 4-12 DQN_SPD_LOSS

从上图 4-12 显示，训练过程中损失值呈现初期快速下降（如从 0.537 降至 0.009），中期存在波动（最高达 0.138），后期趋于稳定但仍有小幅震荡（约 0.004~0.017）。结合 23.22%的避障成功率，表明模型虽能有效降低基础误差（合理动作比例 85.02%），但在复杂场景中仍存在策略不稳定的问题。

从下图 4-13 可以看出，训练过程中合理动作数（RA）波动显著，初期较低（如 1~5 次），中期逐渐提升至 20~40 次，但后期出现极端值（如 196 次），表明模型在基础动作选择上逐步优化，但策略稳定性不足。

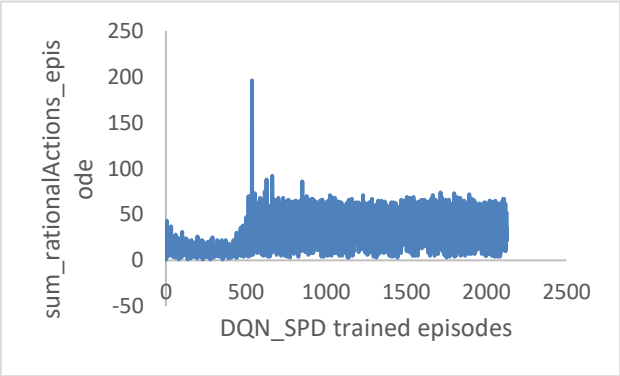

图 4-13 DQN_SPD_RA

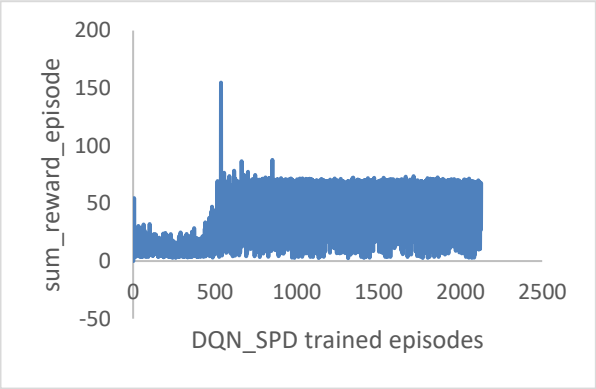

图 4-14 DQN_SPD_reward

图 4-14 表明，通过分析奖励数据可见，初期奖励较低且波动较大，随着训练进行，奖励逐渐上升并趋于稳定，最高达 154.833，表明智能体逐步掌握了避障、车道保持和速度控制策略。后期奖励虽偶有波动，但整体呈上升趋势，验证了 DQN 算法的有效性。

图 4-15 从速度数据来看，初期存在波动（如 36.5 偏高），随着训练推进逐渐稳定在 28-31 区间，后期最高达 31.83 且多数值接近 30-31，表明智能体逐步学会保持合理车速，但偶有逼近环境速度上限的情况，可能需调整奖励函数或网络结构以进一步平滑控制。

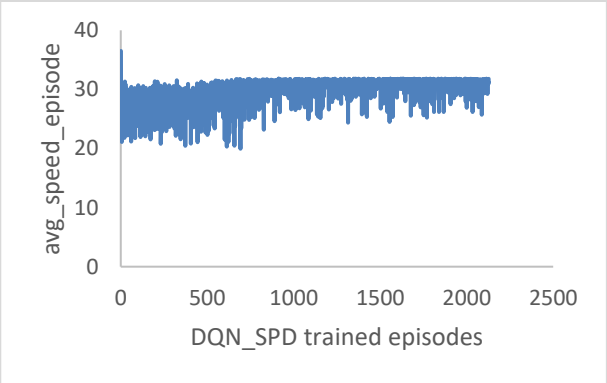

图 4-15 DQN_SPD_speed

4.4 本章小结

本章采用 DQN 算法来进行深度强化学习，分别采用三个算法：含有全部奖励、只含有不合理动作奖励+碰撞奖励 r[0]和只含有速度奖励+碰撞奖励 r[0]，并且分别介绍了它们的算法流程、参数设置，以及仿真实验过后的实验结果分析。

5 基于 DDQN 算法的无人驾驶策略

本章采用 DDQN 算法对上一章的 DQN 算法进行改进，采用两个网络：Q 网络和目标 Q 网络，通过分离动作选择和目标 Q 值计算来减少过估计问题。同样，设定三个对照实验，进行横向与纵向对比。

5.1 DDQN_C(包含全部奖励)

5.1.1 算法流程

(1)初始化环境和参数：

创建高速公路驾驶环境 highway-v0。配置环境参数（车道数、屏幕尺寸、奖励范围、策略频率等）。设置 DQN 超参数（批次大小、记忆容量、训练次数、目标网络更新频率等）。

(2) 定义神经网络结构：

构建 DDQNNet 类，包含 3 层全连接网络（输入层→隐藏层→输出层）。使用 ReLU 激活函数，权重初始化为正态分布。输出层对应 5 种动作（左变道、保持、右变道、加速、减速）。

(3) 初始化 DDQN 智能体：

创建评估网络 eval_net 和目标网络 target_net。定义经验回放缓冲区 memory，使用 namedtuple 存储状态转移（state, action, reward, next_state）。设置优化器（Adam）和损失函数（MSE）。

(4) 探索阶段（填充记忆库）：

重置环境获取初始状态 s；通过 ε-greedy 策略选择动作（随机动作或网络预测动作）；执行动作，获取新状态 s_、奖励 r 和完成标志 done；根据速度、碰撞、动作合理性计算复合奖励；将状态转移(s, a, r, s_)存入记忆库。

(5) 训练阶段（记忆库满后触发）：

从记忆库随机采样 BATCH_SIZE 个样本；

计算评估网络的 Q 值，如式（5.1）所示：

$$q_eval = eval_net(s).gather(a) \tag{5.1}$$

通过评估网络选择目标动作，目标网络计算目标 Q 值如式（5.2）所示：

$$q_next = target_net(s_).gather(max_a) \tag{5.2}$$

计算目标 Q 值如式（5.3）所示：

$$q_target = r + \gamma * q_next \tag{5.3}$$

优化评估网络，最小化 q_eval 与 q_target 的 MSE 损失

软更新目标网络参数如式（5.4）所示：

$$target_param = (1-\tau)*target_param + \tau*eval_param \tag{5.4}$$

(6) 动态调整探索率：

探索率 ε 随训练次数衰减如式（5.5）所示：

$$\varepsilon = 1/(trained_times/50 + 10) \tag{5.5}$$

(7) 结果统计与保存：

记录每轮训练的平均速度、累计奖励、合理动作比例、避障成功率、首次成功避障时间、总训练时间等关键指标。将损失、速度、奖励等数据写入 Excel 文件，使用 Matplotlib 可视化训练结果。

(8) 终止条件：

达到预设的总训练次数 total_train_times。最后输出统计结果并退出程序。

5.1.2 参数设置

本实验基于 DDQN 算法，在 highway-v0 环境中训练驾驶策略。网络采用 20 维输入、1024 神经元隐含层和 5 维输出（对应左变道、保持、右变道、加速、减速）。关键参数包括：学习率 0.001、折扣因子 0.9、经验池容量 4000（200×20）、批次大小 20、总训练步数 80,000。目标网络软更新系数为 0.1，环境配置仿真频率 4Hz，单次训练步数 200，场景含 19 辆周围车辆。奖励机制中高速奖励 0.5，碰撞惩罚-1，非法动作惩罚-2。探索率初始 0.1，按 e=1/(训练次数/50+10)动态衰减，平衡探索与利用。实验通过速度、奖励及避障成功率等指标评估性能，数据存储于 Excel 并通过 Matplotlib 可视化。

5.1.3 实验结果及分析

实验总体结果如图 5-1 所示：

```
-----------------------------------
==== DDQN_C Results ===============
   memory_cnt: 4000
      训练次数: 80000
   总体训练时间: 12986.0
   首次成功用时: 643.0
训练后合理动作数: 72910
   合理动作比例: 91.14 %
   训练奖励总和: 75184.0
     训练后轮次:  1886
   成功避障次数: 497
        成功率: 26.35 %
            e: 0.000621
-------------------------------
```

图 5-1 DDQN_C Results

实验结果分析显示，智能体在 80000 次训练中累计获得 75184 奖励，合理动作占比 91.14%，表明策略较好地遵守车道规则。但避障成功率仅 26.35%，反映复杂交通场景（含 19 辆周围车辆）下避险能力有限。首次成功避障耗时 643 秒，后期探索率降至 0.0006，策略趋于稳定但可能陷入局部最优。训练总时长近 3.6 小时，效率待优化。

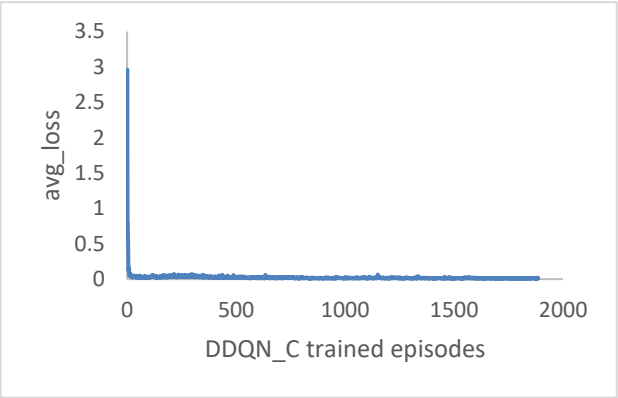

图 5-2 DDQN_C_LOSS

图 5-2 数据表明，训练初期损失值较高（初始 2.96），随后快速下降并稳定在 0.1 以下，但全程存在明显波动（如后期多次回升至 0.05-0.08 区间）。这一趋势反映模型在基础驾驶策略上逐步收敛（合理动作占比 91%），但对复杂避障场景的拟合能力不足（成功率仅 26.35%）。损失波动可能源于高难度场景样本的稀疏性、探索率动态衰减导致的策略震荡，或目标网络软更新系数（0.1）偏高加剧了短期策略偏移。

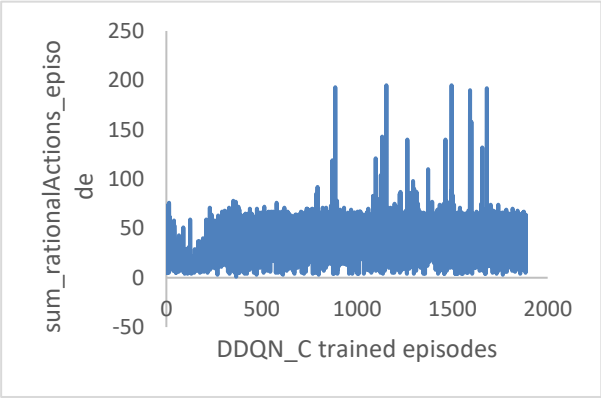

图 5-3 DDQN_C_RA

图 5-3 表明，实验合理动作（RA）数据分布显示，单轮次合理动作数量波动显著（5 至 195 次不等），多数集中在 10-30 次，但存在极端高值（如 195 次），表明策略在部分场景下能高效执行合规动作，但整体稳定性不足。

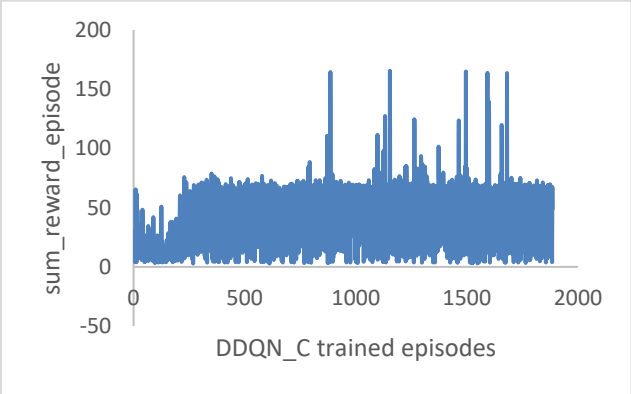

图 5-4 DDQN_C_reward

图 5-4 奖励数据呈现显著波动（单轮次奖励 3 至 190+），表明智能体在简单场景（如直道行驶）表现优异，但复杂避障场景下奖励骤降。高奖励均值（约 40/轮）与低避障成功率（26.35%）的矛盾，揭示奖励函数对关键避障行为激励不足。

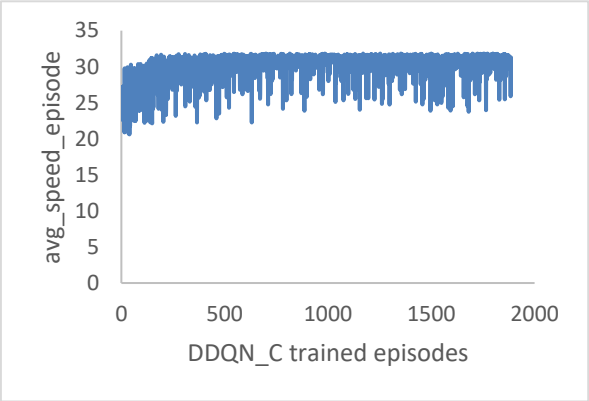

图 5-5 DDQN_C_speed

图 5-5 可以看出，数值分布在 20 到 31.8 之间。从整体趋势看，初期速度多在 20-27 区间波动，随着训练推进，中后期速度显著提升，频繁出现 30 以上的数值，最高达 31.8，表明模型逐渐学会了在保持安全的前提下优化行驶效率。部分波动（如短暂降至 22 左右）可能源于探索动作或复杂场景下的策略调整。

5.2 DDQN_RA（只含有合理动作奖励+碰撞奖励）

5.2.1 算法流程

(1) 初始化环境与参数

创建 Highway 环境并设置参数（车道数、屏幕尺寸、奖励函数参数等）。配置 DDQN 网络参数（学习率、折扣因子、目标网络更新频率等）。初始化 DDQN 网络（评估网络和目标网络）和经验回放池。

(2) 收集经验数据

执行动作选择（基于 ε- 贪婪策略）；与环境交互获取下一个状态、奖励和终止标志；处理奖励（结合环境原始奖励和自定义奖励调整）；将经验 (s,a,r,s') 存入回放池。

(3) 训练阶段

当回放池满时开始训练：从回放池随机采样批次数据；计算目标 Q 值（使用 DDQN 双网络结构）；计算损失函数并执行反向传播；定期软更新目标网络参数。

(4) 记录与评估

记录训练过程中的各项指标（损失、奖励、速度、动作统计等）。当达到预设训练次数后：输出训练结果统计（总训练时间、成功率等）。

(5) 特殊逻辑处理

检测并惩罚不合理动作（如最左车道向左变道），基于环境状态提前终止回合（如完成避障），动态调整探索率 ε（随训练次数增加而减小）。

(6) 循环执行

持续执行上述过程直到满足终止条件（训练次数完成）

5.2.2 参数设置

网络结构设计为输入层维度 20，隐含层包含 1024 个神经元，输出层维度 5，对应环境中的五种驾驶动作（左变道、保持、右变道、加速、减速）。训练过程中，学习率设为 0.001，折扣因子为 0.9，目标网络通过软更新方式同步参数，更新系数为 0.1。经验回放池容量为 4000（200×20），每次训练从池中随机采样 20 条经验进行梯度更新，总训练步数达 80,000 次以充分优化策略。场景中引入 19 辆周围车辆以模拟复杂交通流，奖励函数设计中，高速行驶奖励为 0.5，碰撞惩罚为-1，非法变道动作（如车道边界外的操作）惩罚为-2，通过强化合法动作与惩罚风险行为引导策略收敛。

5.2.3 实验结果及分析

实验总体结果如图 5-6 所示：

```
==== DDQN_RA Results ==============
        memory_cnt: 4000
       训练次数: 80000
    总体训练时间: 12063.0
    首次成功用时: 6314.0
训练后合理动作数: 78600
    合理动作比例: 98.25 %
    训练奖励总和: 78743.0
      训练后轮次:  1035
    成功避障次数: 5
         成功率: 0.48 %
             e: 0.000621
-------------------------------
Waiting for writing excel......
```

图 5-6 DDQN_RA Results

该 DDQN_RA 模型在训练中表现出高合理动作比例（98.25%）与极低避障成功率（0.48%）的矛盾，暗示奖励函数可能过度偏向“动作合规性”而非任务目标。尽管经过 8 万次训练（耗时约 3.35 小时），总奖励较高（78743），但仅 5 次成功避障，且探索率（ε=0.0006）趋近于零，表明策略可能陷入局部最优。

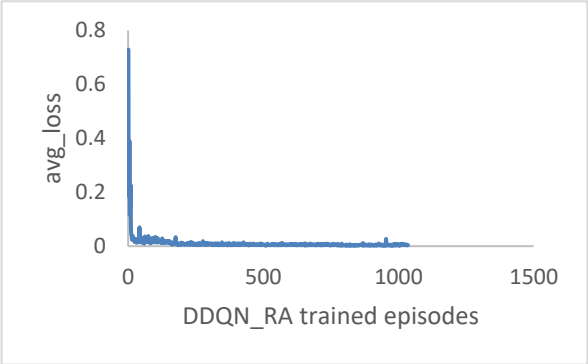

图 5-7 DDQN_RA_LOSS

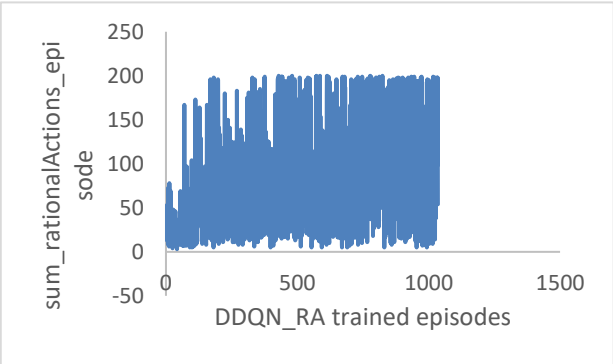

图 5-8 DDQN_RA_RA

图 5-7 表明，损失值从初始的 0.73 逐步降至 0.002 左右，显示模型初期快速收敛，但后期波动频繁且未趋近于零，表明训练后期可能陷入局部最优或存在梯度消失。

图 5-8 可以看出合理动作数在训练初期呈现高波动性（如 21 至 167 次不等），后期逐渐稳定但仍有小幅震荡（如 3 至 9 次），表明模型虽能快速学习合规动作，但未能持续优化策略以实现任务目标。

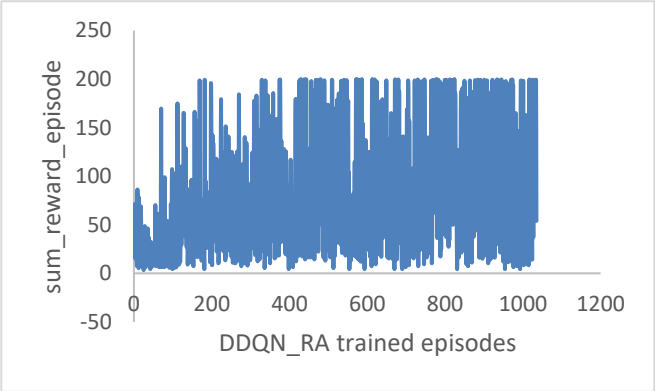

图 5-9 DDQN_RA_reward

图 5-9 表明，奖励值在训练初期波动剧烈（如单步奖励从 0.002 到 0.729），且累积奖励存在极端值（如 200），表明模型偶尔能高效完成任务但整体表现不稳定。奖励分布呈现“长尾”特征（多数步骤奖励低，极少数骤奖励极高）。

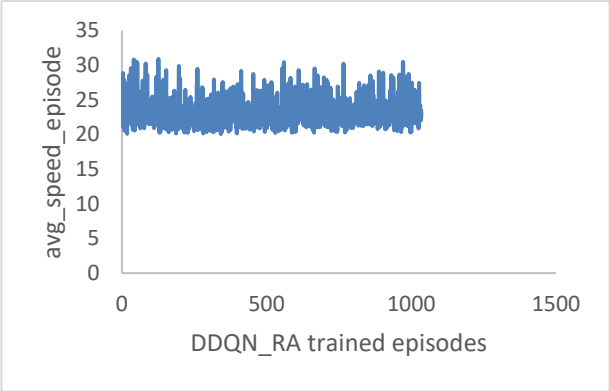

图 5-10 DDQN_RA_speed

图 5-10 可以看出，速度数据整体集中在 20 至 30 区间内，多数值稳定在 21 至 25 之间，表明模型在训练过程中保持了相对稳定的速度表现。数据波动幅度较小（标准差约 2.5），说明速度变化较为平缓，未出现剧烈震荡。整体来看，算法在训练中实现了速

度的均衡控制。

5.3 DDQN_SPD(只含有速度奖励+碰撞奖励)

5.3.1 算法流程

(1) 初始化环境与参数：

创建 Highway 环境并配置车道数、屏幕尺寸、奖励函数等参数。初始化 DDQN 网络（评估网络和目标网络）及其优化器。设置经验回放池容量、批次大小和训练总次数。

(2) 经验收集阶段：

重置环境获取初始状态；根据 ε- 贪婪策略选择动作（探索与利用平衡）；执行动作并获取下一状态、奖励和终止标志；将经验元组 (s,a,r,s') 存入回放池；状态转移：s = s'。

(3) 训练触发条件：

当回放池填满（达到 MEMORY_CAPACITY）时开始训练：设置训练开始标志，记录训练起始时间。

(4) DDQN 训练流程：

从回放池中随机采样批次数据

计算当前 Q 值如式（5.6）所示：

$$q_eval = eval_net\ (s).gather\ (a) \tag{5.6}$$

选择评估网络中最大 Q 值对应的动作如式（5.7）所示：

$$a' = argmax\ (eval_net\ (s')) \tag{5.7}$$

使用目标网络计算 Q 值如式（5.8）所示：

$$q_next = target_net\ (s').gather\ (a') \tag{5.8}$$

目标 Q 值如式（5.9）所示：

$$q_target = r + \gamma * q_next \tag{5.9}$$

计算损失函数：MSE (q_eval, q_target)，反向传播更新评估网络参数

软更新目标网络如式（5.10）所示：

$$target_param = (1-\tau)target_param + \tau eval_param \tag{5.10}$$

(5) 训练后处理：

动态调整探索率 ε（随训练次数增加而减小），记录训练指标（损失、奖励、速度

等），绘制训练过程图表，将数据保存到 Excel 文件。

(6) 终止条件：

达到预设训练次数后退出循环，输出训练结果（成功率、平均奖励等）。

5.3.2 参数设置

本实验基于 DQN 算法，在高速公路驾驶环境（highway-v0）中构建智能体驾驶策略，通过 Double DQN 框架优化网络参数。网络结构设计为三层全连接，输入层维度 20 用于接收环境状态信息，隐含层包含 1024 个神经元以增强特征提取能力，输出层维度 5 对应五种驾驶动作（左变道、保持、右变道、加速、减速）。训练过程中采用学习率 0.001 和折扣因子 0.9 平衡即时与长期收益，目标网络通过软更新机制（系数 0.1）逐步同步主网络参数，经验回放池容量为 4000（200×20），每次训练随机采样 20 组数据进行梯度更新，总训练步数达 80,000 次以确保充分收敛。

环境配置方面，仿真频率设为 4Hz 以匹配控制周期，单次训练最大交互步数为 200 步，场景中设置 19 辆旁车模拟复杂交通流。奖励函数设计注重驾驶效率与安全性：高速行驶奖励 0.5 分，碰撞惩罚-1 分，非法变道等危险动作惩罚-2 分，同时引入速度范围约束机制，将车辆速度动态映射至奖励区间。

5.3.3 实验结果及分析

实验总体结果如图 5-11 所示：

```
==== DDQN_SPD Results ==============
        memory_cnt: 4000
       训练次数: 80000
    总体训练时间: 12079.0
    首次成功用时: 308.0
训练后合理动作数: 66470
    合理动作比例: 83.09 %
    训练奖励总和: 76691.0
      训练后轮次:  1853
    成功避障次数: 537
         成功率: 28.98 %
             e: 0.000621
-------------------------------
Waiting for writing excel......
Writing excel is finished.
```

图 5-11 DDQN_SPD Results

该 DDQN-SPD 模型经过 80000 次训练，总体耗时约 3.4 小时，首次成功避障用时 308 秒，合理动作占比 83.09%，但避障成功率仅 28.98%。训练奖励总和较高（76691），表明模型能有效获取正向反馈，但成功率偏低反映复杂场景下决策能力不足。探索率趋近于零（e=0.0006），后期主要依赖确定性策略，可能需优化奖励机制或网络结构以提升长期决策稳定性。

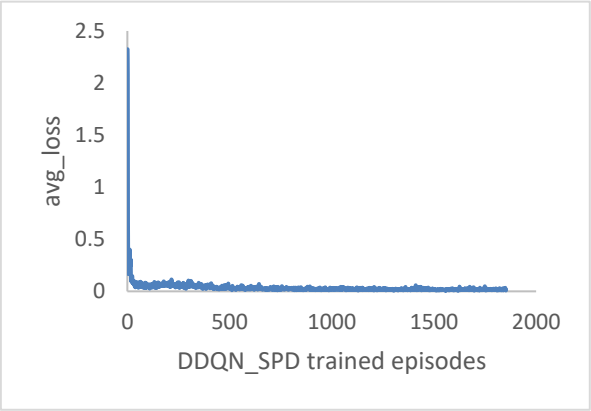

图 5-12 DDQN_SPD_LOSS

图 5-12 可以看出该模型训练初期损失较高（2.32），随着训练逐步下降至 0.005 左右，但波动显著（如中期 0.05-0.1，偶现峰值 0.11），反映策略收敛不稳定。

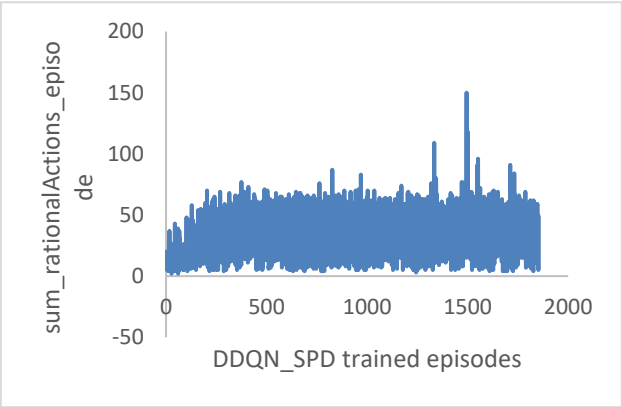

图 5-13 DDQN_SPD_RA

图 5-13 可以看出，合理动作数量波动显著，从个位数（如 6）到极值（如 150），显示模型在不同环境下的决策稳定性不足。多数时段合理动作集中在 20-60 次，但极端低值（如 3）和高值（如 150）并存，表明复杂场景中策略易受干扰。

图 5-14 DDQN_SPD_reward

图 5-14 表明，奖励数据范围广泛，从约 2.6 到 132.4，大部分集中在 10-30 区间，少数高值超过 100。数据波动明显，部分极低值（接近 0）反映初期探索或复杂场景下的低效策略，而较高值表明策略在特定情况下表现优异。

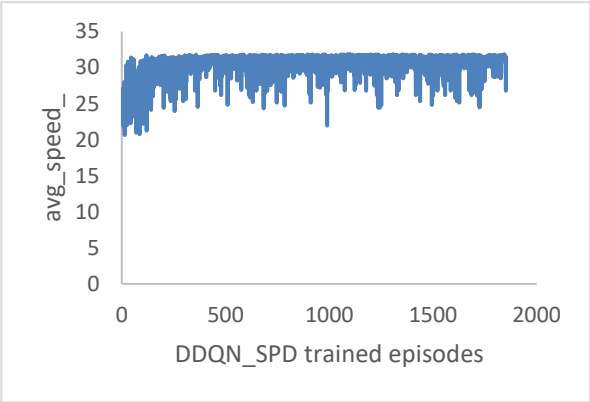

图 5-15 DDQN_SPD_speed

图 5-15 表明，速度数据主要集中在 25 至 31 之间，多数数值接近 30，表明智能体在训练中能有效维持较高速度，符合算法设计中的速度奖励机制。

5.4 本章小结

本章采用 DDQN 算法来进行深度强化学习，分别采用三个算法：含有全部奖励、只含有不合理动作奖励+碰撞奖励 r[0]和只含有速度奖励+碰撞奖励 r[0]，并且分别介绍了它们的算法流程、参数设置，以及仿真实验过后的实验结果分析。

结论

工作总结

本文针对复杂交通场景下无人驾驶决策的实时性与安全性问题，提出了基于深度强化学习的动态决策框架。通过构建 DQN 和 DDQN 算法模型，结合多目标奖励函数设计（碰撞惩罚、速度奖励及合理动作奖励），在 highway-env 仿真环境中验证了不同奖励机制对策略性能的影响。实验结果表明：

（1）奖励函数设计的敏感性：融合速度奖励与合理动作奖励的 DQN_C 模型在驾驶效率上表现优异（合理动作比例达 91.17%），但因避障激励不足，成功率仅为 21.92%；仅依赖合规动作奖励的 DQN_RA 模型虽合规动作比例提升至 98.54%，但避障成功率低至 0.48%，揭示了奖励函数需平衡安全、效率与探索的复杂性。

（2）算法改进的优化效果：DDQN_C 模型通过双网络结构缓解 Q 值高估问题，避障成功率提升至 26.35%，且训练损失波动显著降低，验证了双网络机制在提升策略稳定性中的有效性。

（3）环境交互的挑战：于密集车流的情景里，样本效率低、泛化性不足对模型的避障能力造成限制，说明现有算法在复杂动态交互方面还需进一步改进。

论文通过构建三种奖励组合模型（综合奖励、合规奖励主导、速度奖励主导），在统一交通场景下进行策略训练与测试。对比时，保持算法结构与环境参数一致，仅调整奖励函数的构成权重，观测各模型在驾驶决策中的行为差异（如合规动作频率、避障响应模式、速度稳定性），以此分析不同奖励导向对策略安全性和效率的差异化影响。

本研究具有创新性，体现在系统对比不同奖励机制对决策性能的影响，研发了针对动态交通场景的 DRL 框架，且借助实验揭示了奖励函数与算法结构的协同优化的大致方向，为无人驾驶决策系统的实际应用供给了理论借鉴。

未来期望

虽说本文取得了既定的成果，但在实际场景下依旧有如下改进的空间：

（1）多目标奖励函数的细致化设计：采用可解释性更明朗的奖励分解机制，依靠人类驾驶数据优化权重分配，兼顾安全、效率和舒适体验的需求。

（2）算法鲁棒性的增强：开展对分层强化学习框架的探索，把决策任务分割为高层路径规划和底层动作控制，让状态空间的复杂度降低；结合采用元强化学习技术，依靠历史经验加速新场景策略的转移。

（3）多智能体协同决策：组建车联网环境下的多智能体 DRL 模型，经由车辆与基础设施的信息共享达成协同避障与交通流分配的优化，提高系统层面的决策效率。

（4）样本效率与泛化能力提升：引入生成式 AI 技术（如 GAN、VAE）动态生成高保真仿真数据，缓解真实数据稀缺问题；结合离线强化学习方法，利用历史驾驶日志预训练策略模型。

（5）实际部署的可行性验证：在真实路测平台中验证算法性能，解决传感器噪声、通信延迟等现实约束；开发安全验证框架（如形式化验证），确保决策策略符合交通法规与伦理规范。

未来研究可进一步融合量子计算加速 DRL 训练过程，探索边缘计算设备上的轻量化部署方案，推动无人驾驶决策系统从仿真验证走向大规模实际应用。

参考文献

[1] 杜牵.基于风险评估和深度强化学习的自动驾驶决策方法研究[D].齐鲁工业大学,2024

[2] 张俊阳.基于深度强化学习的端到端无人驾驶决策算法研究[D].电子科技大学,2022

[3] 王一权.基于深度强化学习的无人驾驶决策研究[D].天津理工大学,2022

[4] 孙楠.基于深度强化学习算法与粒子群优化的无人驾驶决策研究[D].大连交通大学,2023

[5] 姜鳙祝.基于DDPG算法的无人驾驶决策算法研究[D].哈尔滨工业大学,2021

[6] 陈建文.基于深度强化学习的无人驾驶路径规划算法研究[D].河北工业大学,2021

[7] 左思翔.基于深度强化学习的无人驾驶智能决策控制研究[D].哈尔滨工业大学,2018

[8] 刘佳琦.基于深度强化学习的无人驾驶感知与决策研究[D].天津理工大学,2021

[9] 张泽功.基于深度强化学习算法的智能驾驶决策技术研究[D].桂林电子科技大学,2020

[10]韩慧.基于深度学习和强化学习的目标跟踪算法研究[D].南京理工大学,2020

[11]曾磊.个性化风险评估驱动的无人车换道决策规划方法[D].湖南大学,2023

[12]刘学高.基于深度强化学习的自动驾驶拟人化跟驰决策算法研究[D].西南大学,2023

[13]马兆伟，牛轶峰，王菖，沈林成作.基于学习的无人机感知与规避[M].北京：国防工业出版社,2023

[14] 付一豪，鲍泓，梁天骄，付东普，潘峰.基于视觉DQN的无人车换道决策算法研究[J].传感器与微系统,2023,第42卷(10): 52-55

[15] Cui B ,Chen Y ,Hong X , et al.Research on Path-Following Technology of a Single-Outboard-Motor Unmanned Surface Vehicle Based on Deep Reinforcement Learning and Model Predictive Control Algorithm[J].Journal of Marine Science and Engineering,2024,12(12):2321-2321.

[16] Demir K ,Tumen V ,Kosunalp S , et al.A Deep Reinforcement Learning Algorithm for Trajectory Planning of Swarm UAV Fulfilling Wildfire Reconnaissance[J].Electronics,2024,13(13):2568-2568.

[17] Li N ,Chen P .Research on a Personalized Decision Control Algorithm for Autonomous Vehicles Based on the Reinforcement Learning from Human Feedback Strategy[J].Electronics,2024,13(11):2079-9292.

[18] Kobbi I ,Benamirouche A ,Tadjine M .Enhancing pose estimation for mobile robots: A comparative analysis of deep reinforcement learning algorithms for adaptive Extended Kalman Filter-based estimation[J].Engineering Applications of Artificial Intelligence,2025,150110548-110548.

[19] Ivanovic S ,Kebir E M .CNRein: an evolution-aware deep reinforcement learning algorithm for single-cell DNA copy number calling[J].Genome Biology,2025,26(1):87-87.

[20] Dong X ,Wan G ,Zeng P .A heuristic-assisted deep reinforcement learning algorithm for flexible job shop scheduling with transport constraints[J].Complex & Intelligent Systems,2025,11(5):210-210.

[21] Shen Y ,Liao Z ,Chen D .Differential Evolution Deep Reinforcement Learning Algorithm for

Dynamic Multiship Collision Avoidance with COLREGs Compliance[J].Journal of Marine Science and Engineering,2025,13(3):596-596.

[22] Yucheng L ,Lingyun Q ,Kainan S , et al.Development and validation of a deep reinforcement learning algorithm for auto-delineation of organs at risk in cervical cancer radiotherapy[J].Scientific Reports,2025,15(1):6800-6800.

[23] Chen Z ,Ding R ,Zhou Q , et al.Research on coordinated control of electro-hydraulic composite braking for an electric vehicle based on the Fuzzy-TD3 deep reinforcement learning algorithm[J].Control Engineering Practice,2025,157106248-106248.

[24] Liu L ,Jing T ,Li W , et al.Meta learning-based deep reinforcement learning algorithm for task offloading in dynamic vehicular network[J].Engineering Applications of Artificial Intelligence,2025,143109966-109966.

[25] Qin Y ,Chen J ,Jin L , et al.Task offloading optimization in mobile edge computing based on a deep reinforcement learning algorithm using density clustering and ensemble learning[J].Scientific Reports,2025,15(1):211-211.

[26] He Y ,Hu R ,Liang K , et al.Deep Reinforcement Learning Algorithm with Long Short-Term Memory Network for Optimizing Unmanned Aerial Vehicle Information Transmission[J].Mathematics,2024,13(1):46-46.

[27] Cui B ,Chen Y ,Hong X , et al.Research on Path-Following Technology of a Single-Outboard-Motor Unmanned Surface Vehicle Based on Deep Reinforcement Learning and Model Predictive Control Algorithm[J].Journal of Marine Science and Engineering,2024,12(12):2321-2321.

[28] [11]Demir K ,Tumen V ,Kosunalp S , et al.A Deep Reinforcement Learning Algorithm for Trajectory Planning of Swarm UAV Fulfilling Wildfire Reconnaissance[J].Electronics,2024,13(13):2568-2568.

致　谢

论文的工作就要结束了，四年大学生涯眼看就要收尾了。

我对指导老师李爱萍教授表达感激，感谢在毕业设计跟论文编写的过程里面，李老师对我的帮扶与指导，让我得以顺利完成这项任务，她不但对我们的论文选题以及进度很留意，还会主动为我们消除疑惑。

感谢这一路朋友、家人对我的关心支持，在你们体贴入微的关心下，我才可以有信心、有毅力去结束毕设任务。

路漫漫其修远兮，吾将上下而求索。在未来的探索道路上，期望自己能守住初心，也盼望大家都能愿望成真。

外文原文

Research on Path-Following Technology of a Single-Outboard-Motor Unmanned Surface Vehicle Based on Deep Reinforcement Learning and Model Predictive Control Algorithm

Abstract

Path following is one of the key technologies for unmanned surface vehicles (USVs). This paper proposes a path-tracking control method for a single-outboard-motor USV based on a Deep Deterministic Policy Gradient (DDPG) algorithm and model predictive control (MPC) algorithm. Initially, the motion model and outboard motor model of the USV are analyzed. Subsequently, simulation and real ship experiments provide a comprehensive performance comparison between the proposed DDPG-MPC method and the traditional ALOS-PID method. The results indicate that for straight path tracking, the DDPG-MPC algorithm achieves 37% and 21% reductions in the average cross error and heading angle error, respectively, compared to the ALOS-PID algorithm. The real ship experiments further validate the DDPG-MPC algorithm's advantages in real-world environments. Specifically, under disturbances like wind, waves, and currents, the maximum cross error of the DDPG-MPC algorithm is one-third of the ALOS-PID algorithm. Additionally, the DDPG-MPC algorithm sustains a higher and more stable longitudinal velocity over extended periods, while the ALOS-PID algorithm shows greater instability and variability. Overall, the findings confirm the feasibility and effectiveness of the proposed approach, highlighting its potential for enhancing path-tracking control performance in single-outboard-motor USVs.

Keywords: path following; unmanned surface vehicle; deep reinforcement learning; deep deterministic policy gradient; model predictive control

1 Introduction

Unmanned surface vehicles (USVs) are increasingly valued for their intelligence, ma-neuverability, and adaptability in various applications such as water patrols, hydrological surveys, maritime operations, rescue missions, and military usage [1–4]. Path following, a crucial technology for the autonomous navigation of USVs, focuses on maximizing effi-ciency and ensuring safety by maintaining adherence to a planned path within accepted error margins, as opposed to trajectory tracking, which involves strict timing at specific points [5–7]. Path-following technology principally involves path guidance and heading control components [8–10]. Among the many methods studied, Line-of-sight (LOS) and vector field (VF) methods are widely used [11, 12]. These approaches facilitate the control of underactuated vessels by transforming the path-following challenge into a heading management task through well-developed guidance laws.

Several advancements have enhanced the LOS method's efficacy. Wang et al. [13] introduced a Predictor-Based Fixed-Time LOS guidance law tailored for USVs navigating in unknown disturbances, featuring global fixed-time convergence that guarantees quick stabilization and robust performance, regardless of the initial conditions. Liu et al. [14] developed an adaptive LOS (ALOS) that finely adjusts to changes in sideslip, maintaining trajectory precision under varying conditions. Zhou et al. [15] enhanced this approach with their integral–differential LOS (IDLOS), which incorporates adaptive elements to speed convergence and minimize overshoot, optimizing response times and control pre-cision. Further advancements came from Tong et al. [16], who proposed a finite-time, error-constrained LOS suitable for navigating curved paths under uncertain circumstances, expanding the versatility of LOS applications. Complementarily, Fossen et al. [17] crafted a nonlinear ALOS method that effectively counters environmental drift, such as wind and cur-rents, ensuring uniform semi-global exponential stability (USGES) under linear trajectory conditions. In comparative analysis, ALOS guidance [16] stands out in dynamic environ-ments due to its rapid adaptability and robust stability, while the ALOS law [17] offers finite-time convergence and cross-track error constraints, ideal for underactuated

systems facing unpredictable challenges. On the other hand, Wang's Predictor-Based Fixed-Time LOS [13] provides a resilient framework ensuring swift and precise control. Nonetheless, despite higher complexity and computational demands, these advanced LOS methods excel in scenarios requiring an immediate response and exceptional accuracy, with ALOS proving particularly effective in resource-limited settings. Hong et al. [18] refined LOS by integrating a variable "acceptance circle" radius to enhance flexibility during waypoint transitions, offering a significant improvement over traditional methods. In contrast, VF methods, developed initially for micro aerial vehicles, have been adapted for USVs [19–21], with comparative studies suggesting these methods provide excellent path adherence but may cause yaw instability [22]. Collectively, these adaptations balance robustness, convergence speed, and computational efficiency, pushing forward the development of unmanned ship path-following technology. However, the current research landscape in this domain is not without its limitations. The existing research and practical application of unmanned surface ship heading guidance laws based on the LOS method and the VF method have inherent defects [23]. The LOS method grapples with the challenge of striking a balance between the response time and steady-state error in forward distance settings [24]. Moreover, when transitioning between different desired paths, the convergence speed of the associated controller cannot be assured [25]. Conversely, the VF method is plagued by rudder oscillation in the horizontal direction [26].

Recent research [27–31] has newly incorporated deep reinforcement learning into USV path-following strategies, highlighting novel advancements in control methodologies. Zhao et al. [6] developed an enhanced deep Q-network (DQN) for the simplified control of three-degree-of-freedom USV models, unveiling significant potential for adaptive learning in maritime navigation scenarios. Zhong et al. [32] introduced a novel control approach utiliz-ing a Deep Deterministic Policy Gradient (DDPG) algorithm designed around a composite state space and dynamic reward function, aimed at reducing the impacts of large inertia and state lag on RL agents' training. Han et al. [33] has developed a system using distributed deep reinforcement learning and adaptive neural networks for straight-line path

tracking and formation control in USVs, employing radial basis function neural networks (RBF NNs) to approximate the hydrodynamic forces and unknown external disturbances affecting the vehicles, demonstrating high precision in tracking control through simulation results. Furthermore, Zhu et al. [34] has proposed an intelligent path-tracking control strategy for underactuated USVs with unknown model parameters, combining LOS guidance and Q-learning to enhance path-following accuracy under uncertain dynamics. These advances underscore the growing versatility and challenge of applying deep reinforcement learning, showcasing its potential in achieving sophisticated maritime navigational tasks while high-lighting inherent limitations such as computational complexity, data requirements, and response time to dynamic environmental changes.

However, a considerable part of the research still relies on differential twin propeller thrusters configured at the stern and cannot be directly converted to a single-outboard-motor setup [27–31], which highlights the need for specific solutions for different propulsion types. This paper introduces a novel path-following control method for a single-outboard-motor USV that combines deep reinforcement learning with model predictive control. This approach addresses the shortcomings of traditional LOS and VF approaches and is tailored to the specific propulsion characteristics of a single-outboard-motor USV. By implementing deep reinforcement learning for path tracking in a single-outboard-motor USV mission, we significantly improve the path-following capabilities. At the same time, our reinforcement learning method DDPG-MPC also improves the versatility of reinforcement learning-based path tracking in complex ocean environments, contributing innovation to autonomous USV path tracking. By employing this cutting-edge strategy, we extend the practical applicability of reinforcement learning to different operating conditions and improve the resilience and adaptability of USVs to a wider range of environmental challenges.

2 Kinematic Modeling

2.1 Kinematic Modeling USV

The kinematic modeling of USVs involves multiple factors such as thruster forces and moments, viscous hydrodynamics and external disturbances, and has highly nonlinear and variable coupling characteristics, making it a very complex process [35]. It needs to balance two aspects. Firstly, in order to avoid the model being too complex and unable to guarantee real-time performance in subsequent control operations, reasonable assumptions and simplifications need to be made during modeling. The second is to avoid neglecting some key system characteristics of the model when simplifying it.

The kinematic modeling of unmanned surface vehicles (USVs) typically involves two coordinate systems: the world coordinate system $OX_wY_wZ_w$ *and* the body-fixed coordinate system $OX_bY_bZ_b$ *[36]*, as illustrated in Figure 1. In this paper, the world coordinate system is defined as the NED (North-East-Down) coordinate system. Within this system, the six degrees of freedom (DOF) motion vector of the USV can be expressed as $\eta = [x, y, z, \phi, \theta, \psi]^T$, where *x*, *y*, *and* z represent the *displacements* along the Xw, Yw, *and Zw* axes, respectively, and ϕ, θ, and ψ *denote* the *roll*, *pitch*, and *yaw angles*, corresponding to rotations about the *Xw*, Yw, and Zw axes.

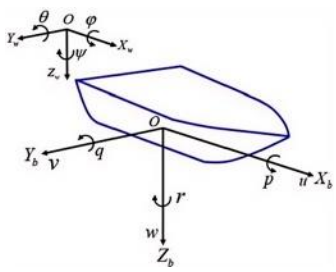

Figure 1. Coordinate system for kinematic modeling of USV.

In the body-fixed coordinate system, this paper defines the origin *O* as the center of gravity of the USV, X_b as the forward direction, Y_b as the starboard direction, and Z_b as the downward direction of the USV. The rotations around the X_b, Y_b, and Z_b axes produce roll rate *p*, pitch rate *q*, and yaw rate *r*, respectively. The translations along the X_b, Y_b, and Z_b axes generate surge speed *u*, The sway speed is *v*, and the heave speed is *w*. In the

body-fixed coordinate system, the velocity of the USV can be represented by the vector $\upsilon = [u, v, w, p, q, r]^T$.

In the study of USV dynamics, more emphasis is placed on the planar motion state of the USV. Therefore, the motions in roll, pitch, and heave are generally ignored. As a result, the motion of the USV can be represented by a three-degree-of-freedom motion model based on Lagrangian mechanics, as shown in Equation (1).

$$\boldsymbol{M}\dot{\boldsymbol{v}} + \boldsymbol{C}(\boldsymbol{v})\boldsymbol{v} + \boldsymbol{D}(\boldsymbol{v})\boldsymbol{v} = \boldsymbol{\tau} \tag{1}$$

Here, $\boldsymbol{M}$ stands for the inertia matrix, $\boldsymbol{C}(\upsilon)$ represents the Coriolis and centripetal force matrix, and $\boldsymbol{D}(\upsilon)$ denotes the hydrodynamic damping matrix. The above-mentioned terms can be computed by the following formulas:

$$\boldsymbol{M} = \begin{bmatrix} m - X_{\dot{u}} & 0 & 0 \\ 0 & m - Y_{\dot{v}} & mx_g - Y_{\dot{r}} \\ 0 & mx_g - Y_{\dot{r}} & I_z - N_{\dot{r}} \end{bmatrix} \tag{2}$$

$$\boldsymbol{C}(\boldsymbol{v}) = \begin{bmatrix} 0 & 0 & -(m - Y_{\dot{v}})v - (mx_g - Y_{\dot{r}})r \\ 0 & 0 & (m - X_{\dot{u}})u \\ (m - Y_{\dot{v}})v + (mx_g - Y_{\dot{r}})r & -(m - X_{\dot{u}})u & 0 \end{bmatrix} \tag{3}$$

$$\boldsymbol{D}(\boldsymbol{v}) = \begin{bmatrix} -X_u & 0 & 0 \\ 0 & -Y_v & -Y_r \\ 0 & -N_v & -N_r \end{bmatrix} \tag{4}$$

where X_u, Y_v, Y_r, N_r, X_u, Y_v, Y_r, N_v, and N_r are the hydrodynamic coefficients of the USV.

$\boldsymbol{v}$ represents the three-degree-of-freedom velocity vector of the USV in the body-fixed coordinate system, as shown in Equation (5).

$$\boldsymbol{v} = \begin{bmatrix} u & v & r \end{bmatrix}^T \tag{5}$$

τ represents the force exerted by the propeller on the USV, as shown in Equation (6).

$$\boldsymbol{\tau} = \begin{bmatrix} \tau_u & \tau_v & \tau_r \end{bmatrix}^T \tag{6}$$

Among them, τu represents the longitudinal thrust, τv the lateral thrust, and τr the yaw moment. These three quantities can be calculated using Equation (7).

$$\begin{cases} \tau_u = T\cos\delta \\ \tau_v = T\sin\delta \\ \tau_r = x_l T\sin\delta \end{cases} \tag{7}$$

Where T represents the thrust of the outboard motor, δ represents the thrust angle of the outboard motor, and xl represents the distance from the rotation axis of the outboard motor to the center of rotation of the USV.

The USV driven by a single outboard engine has only two control variables: the thrust angle and thrust of the outboard engine. However, the USV needs to control three degrees of freedom, which makes it an underactuated system. Based on the underactuated characteristics of a single-outboard-motor USV, the following definitions are made in the paper [37, 38]: $m_{11} = m - X_{\dot{u}}$, $m_{22} = m - Y_{\dot{v}}$, $m_{23} = mx_g - Y_{\dot{r}}$, $m_{32} = mx_g - N_{\dot{v}}$, $m_{33} = I_z - N_{\dot{r}}$, $d_{11} = -X_u$, $d_{22} = -Y_v$, $d_{23} = -Y_r$, $d_{32} = -N_v$, and $d_{33} = -N_r$. So, Equation (1) can be transformed into Equation (8).

$$\begin{bmatrix} m_{11}\dot{u} \\ m_{22}\dot{v} + m_{23}\dot{r} \\ m_{32}\dot{v} + m_{33}\dot{r} \end{bmatrix} + \begin{bmatrix} d_{11}u - m_{22}vr - m_{23}r^2 \\ d_{22}v - d_{23}r + m_{11}ur \\ m_{22}vu + m_{23}ru + d_{32}v - m_{11}uv + d_{33}r \end{bmatrix} = \begin{bmatrix} \tau_u \\ 0 \\ \tau_r \end{bmatrix} \tag{8}$$

Assuming that the center of gravity of the hull coincides with the geometric center and that the USV is symmetric in the fore–aft and port–starboard directions, we have $x_g = 0$, $Y_{\dot{r}} = 0$, $N_{\dot{v}} = 0$, $Y_r = 0$, and $N_v = 0$, allowing Equation (8) to be simplified to Equation (9).

$$\begin{cases} \dot{u} = \dfrac{m_{22}}{m_{11}}vr - \dfrac{d_{11}}{m_{11}}u + \dfrac{1}{m_{11}}\tau_u \\ \dot{v} = -\dfrac{m_{11}}{m_{22}}ur - \dfrac{d_{22}}{m_{22}}v \\ \dot{r} = \dfrac{m_{11} - m_{22}}{m_{33}}uv - \dfrac{d_{33}}{m_{33}}r + \dfrac{1}{m_{33}}\tau_r \end{cases} \tag{9}$$

2.2 Kinematic Modeling of the Outboard Motor

As shown in Equation (7), the longitudinal thrust, lateral thrust, and steering moment exerted by the outboard motor on the USV are determined by the thrust T and thrust angle δ of the outboard motor. Therefore, it is necessary to model the thrust model and servo model of the single outboard motor.

2.2.1. The Outboard Motor Model

The outboard motor has the advantage of being easy to retract, allowing it to be stowed away when an amphibious USV comes ashore, thereby avoiding interference with the ground. This makes it suitable as a propulsion system for amphibious USVs. The thrust generated by the outboard motor is determined by the propeller's rotational speed, and its thrust mathematical model is given by Equation (10) [39]:

$$T = (1 - t_P)\rho n^2 D_P^4 K_T(J_p) \tag{10}$$

Where ρ represents the density of seawater, n denotes the rotational speed of the propeller, and DP indicates the diameter of the propeller. Additionally, tP stands for the reduction coefficient, KT (Jp) represents the thrust coefficient, and Jp indicates the advance coefficient. The calculation methods for these coefficients are described in detail below.

The reduction coefficient tP can be approximated using the Hallrot empirical formula [40], which is suitable for single-propeller propulsion systems, as shown in Equation (11).

$$t_P = 0.001979\frac{L}{(B - BC_{p1})} + 1.0585C_{10} - 0.000524 - 0.1418\frac{D_P^2}{Bd} + 0.0015C_{stern} \tag{11}$$

where L represents the waterline length, B denotes the beam of the vessel, and d indicates the draft depth. And Cp1 can be calculated using Equation (12).

$$C_{p1} = 1.45C_P - 0.315 - 0.225L_{cb} \tag{12}$$

where CP represents the prismatic coefficient and Lcb denotes the position of the forward half-length center of buoyancy.

C10 represents a coefficient related to the length-to-beam ratio, which can be calculated using Equation (13). Cstern denotes a coefficient related to the stern shape of the vessel, which can be determined using Equation (14).

$$\begin{cases} C_{10} = B/L & L/B > 5.2 \\ C_{10} = 0.25 - \dfrac{0.003328402}{B/L - 0.134615385} & L/B \leq 5.2 \end{cases} \tag{13}$$

$$\begin{cases} C_{stern} = -10 \quad V - Shaped\ Hull \\ C_{stern} = 0 \quad Conventional\ Hull \\ C_{stern} = +10 \quad U - Shaped\ Hull\ with\ a\ Hogner\ Stern \end{cases} \tag{14}$$

The thrust coefficient KT can be calculated using multiple regression analysis [41], as illustrated in Equation (15).

$$K_T = \sum_{i=0}^{n1} \sum_{j=0}^{n2} A_{ij} \left(\frac{P}{D} \right)^i J_P^j \tag{15}$$

where JP = represents the advance coefficient, and wp denotes the wake coefficient, nDp which can be determined using the Bappmiller formula, as shown in Equation (16).

$$w_p = 0.165 C_b^x \sqrt{\frac{\sqrt[3]{\nabla}}{D_p}} - \Delta w \tag{16}$$

where ∇represents the displacement volume, x is a coefficient related to the propeller installation position, as shown in Equation (17), and Δw represents the correction value for the wake coefficient, which can be expressed using Equation (18).

$$\begin{cases} x = 1 & \textit{Propeller Positioned on the Centerline} \\ x = 2 & \textit{Propeller Positioned on the Side} \end{cases} \tag{17}$$

$$\Delta w = \begin{cases} 0.1(F_n - 0.2), & F_n > 0.2 \\ 0, & F_n \leq 0.2 \end{cases} \tag{18}$$

where Fn = V/p gL is the Froude number and g is the gravitational acceleration.

2.2.2 The Servo Model

The outboard engine servo model can be represented as a first-order inertial compo-nent, which aligns with the approach by M. T. Meziou, J. Ghommam, and N. Derbel in their paper [42], as shown in Equation (19).

$$\dot{\delta} = -\frac{1}{T_d}\delta + \frac{1}{T_d}\delta_d \tag{19}$$

Where Td represents the time coefficient, δ denotes the current propulsion angle, and δd indicates the target propulsion angle.

3 Path-Following Control Algorithm

This paper presents a design strategy that divides the path-following control frame-work of a single-outboard-motor USV into two key components: Path-following guidance law and heading control. To overcome the limitations of conventional algorithms, it introduces a novel method integrating DDPG with MPC, as illustrated in Figure 2. This method calculates the desired heading angle ψ_d from a pre-planned path and uses an MPC-based heading controller to adjust the USV's heading angle, enabling effective path following.

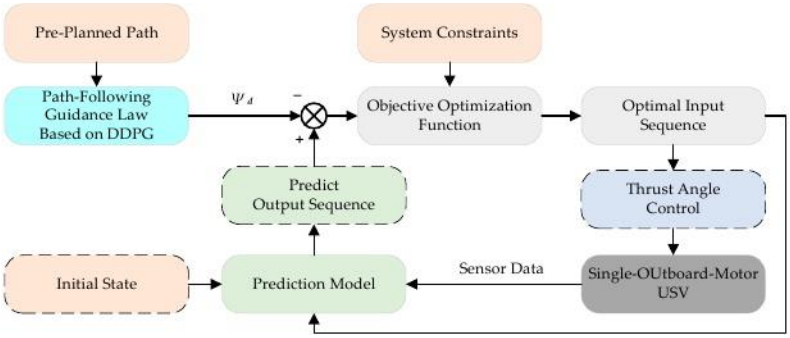

Figure 2. Algorithm framework for path-following control of single-outboard-motor USV.

3.1 Path-Following Guidance Law Based on DDPG Algorithm

3.1.1 Design of the Markov Decision Process

In reinforcement learning, constructing a Markov decision model is a key element. The algorithm's efficiency and convergence heavily rely on accurately defining the state space, action space, and reward function [43]. Hence, to develop a high-performance guidance law for USV path following, it is crucial to design a suitable Markov decision model tailored to the specific requirements of single-outboard-motor USVs.

The state space is defined as follows:

$$S = [u, r, \beta, \dot{\beta}, \varepsilon, \dot{\varepsilon}, \Delta\psi_d^p] \tag{20}$$

where u represents the longitudinal velocity of the USV, r denotes the USV's pitch rate, β is the deviation between the USV's heading angle and the desired path angle, ε is the cross error between the desired path and the USV's real-time position, and Δψd is the action taken in the previous time step, which is the desired change in the heading angle of the USV.

In path-following control, greater emphasis is placed on the longitudinal velocity of the

USV, while lateral drift velocity is often overlooked. To ensure the path-following guidance law can generate a real-time optimal desired heading angle based on the USV's longitudinal velocity u, This study incorporates the longitudinal velocity into the state space. The state space includes the deviation β between the current heading angle and the desired path angle, as well as the cross-track error between the desired path and the USV's real-time position, reflecting the discrepancy between the actual and desired positions. Additionally, to evaluate if the USV's state changes align with the trend of reducing tracking errors, the state space also integrates the rate of change in the deviation between the heading and path .

angles β, the rate of change in the cross-track error, and the action taken in the previous time step Δψd p.

The action space is defined as follows:

$$\boldsymbol{A} = [\Delta\psi_d] \tag{21}$$

$\Delta_{\psi d}$ represents the change in the desired heading angle of the USV, and $\Delta_{\psi d} \in$?– π /2, π /2 ? such that the USV's desired heading angle $_{\psi d}$ can be calculated using the formula $_{\psi d} = \psi + \Delta_{\psi d}$, where ψ denotes the actual heading angle of the USV.

The reward function is defined as follows:

$$r = \begin{cases} r_\varepsilon + r_\beta + r_{\Delta\psi_d} & |\beta| < \frac{\pi}{2} \\ r_\beta & |\beta| \geq \frac{\pi}{2} \end{cases} \tag{22}$$

The definitions of each term are as follows:

$$r_\varepsilon = e^{-k_\varepsilon|\varepsilon|} \tag{23}$$

$$r_\beta = e^{-k_\beta|\beta|} \tag{24}$$

$$r_{\Delta\psi_d} = e^{-k_{\Delta\psi_d}|\Delta\psi_d|} \tag{25}$$

Where kε, kβ, and kΔψd are constants, calibrated through multiple experiments, with kε = 0.7, kβ = 4.82, and kΔψd = 0.3.

Equation (23) describes the reward function for the cross error between the actual position of the unmanned boat and the desired path. As shown in Figure 3a, the function has the following characteristics: when the cross error is 0, the value reaches 1; as the absolute

value ofincreases, the reward value decreases rapidly. However, when the error increases to 8 m (approximately the length of a boat), the reward valueis almost close to 0. This design encourages the agent to actively minimize the cross error during the interaction with the environment in the form of an exponential function, thereby achieving more accurate path tracking.

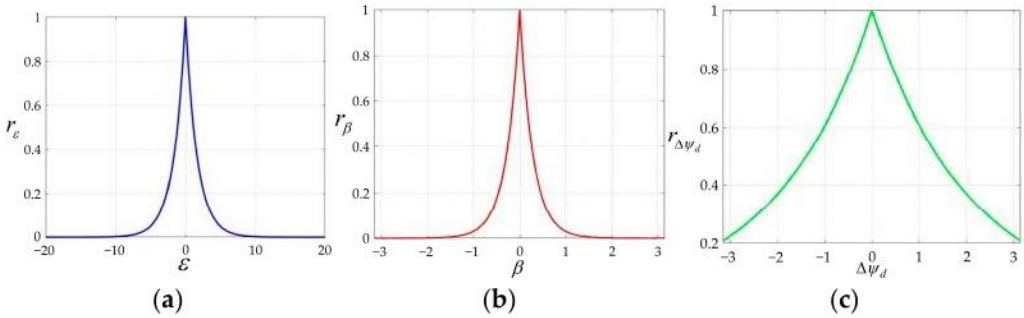

Figure 3. The reward function. **(a)** The reward function r ε associated with the cross-track error ε between the actual position of the USV and the desired tracking path. **(b)** The reward function r β related to the deviation β between the heading angle of the USV and the desired path angle. **(c)** The reward function $r\Delta_{\psi d}$ related to the change in the desired heading angle $\Delta_{\psi d}$ of the USV.

Equation (24) represents the reward function associated with the deviation between the USV's heading angle and the desired path angle. The graph of this function is depicted in Figure 3b. When the angle deviation is 0, the value of reaches its maximum of 1. To prevent the USV from veering off the desired path, that is, when, the value of remains close to 0.

Equation (25) defines the reward function rΔψd associated with changes in the USV's desired heading angle Δψd. As illustrated in Figure 3c, this function is meticulously designed to minimize sudden changes in the desired heading direction. This ensures a smooth and continuous adjustment process. enhancing the stability and precision of the USV's navigation path. This approach ultimately contributes to more efficient and reliable path tracking.

When the heading angle and the desired path angle have a large difference, the heading angle should be adjusted as soon as possible to prevent the deviation between the actual position of the USV and the desired tracking path from increasing further. Therefore, r = rβ is selected; when |β| < π/2, all three reward functions are taken into account, and r = rε + rΔψd +

rβ is chosen.

3.1.2 Implementation of the DDPG Algorithm

The DDPG algorithm is a type of deep reinforcement learning method built on theActor–Critic framework [44]. Its structural principles are shown in Figure 4.

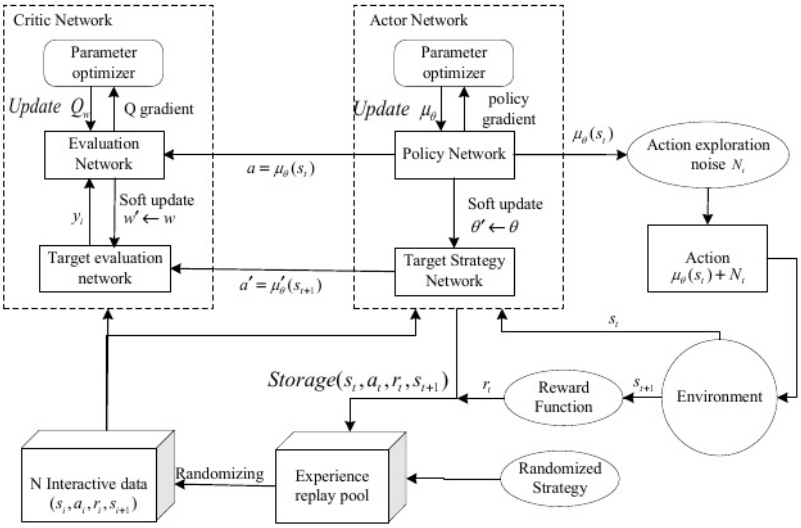

Figure 4. Framework of the DDPG algorithm.

The DDPG agent is composed of a policy network and a value network. The policy network, known as the Actor Network, takes the state vector ***st*** *as* input and generates the action vector μ θ (st) in accordance with the action policy, where θ represents the weights *of* the Actor Network. Structurally, the Actor Network comprises an input layer, two hidden layers, and an output layer. The input layer's size aligns with the state vector's dimension, while the output layer matches the action vector's dimension. ReLU is used as the activation function in the hidden layers, and the output layer applies the tanh function to normalize the action values, which are then scaled to the actual action range.

The value network, referred to as the Critic Network, takes the state vector st and the action vector at generated by the Actor Network as inputs and outputs the action value Qω(st, at), where ω represents the weights of the Critic Network. The state vector is processed through two hidden layers. while the action vector passes through one hidden layer. These outputs are then combined via tensor addition, followed by an additional hidden layer and an output layer. ReLU serves as the activation function across all hidden layers. The training process of the DDPG agent includes the following steps:

1. Randomly initialize the Actor Network μ(s| θ) and Critic Network Q(s, a| ω); initialize the target network's actor ω' ← ω and θ' ← θ ; initialize the experience replay buffer R.

2.To start a new episode, randomly initialize the USV's state in the environment as the initial state s1. At each time step, the current state vector st and the action policy μθ(st) are combined with exploration noise Nt to generate the action vector at = μθ(st) + Nt, which defines the desired heading angle of the USV. The heading controller then tracks this angle. and the environment provides the updated state vector st+1. Meanwhile, rewards are computed using environmental data, such as cross-track and heading angle errors. Interaction data (st, at, rt, st+1) generated at each time step are saved in the experience replay buffer R.

3.Randomly sample N interaction data from the experience replay buffer R to update the networks. Update the Critic Network by minimizing the loss function L = N1 ∑i (yi − Qω(si, ai))2, where yi = ri + γQ 'ω'(si+1, μ'θ'(si+1)) represents the target value output by the target Critic Network. Update the Actor Network based on the sampled gradients, with the gradient formula given as follows:

$$\nabla_\theta \mu|_{s_t} \approx \frac{1}{N}\sum_i \nabla_a Q_\omega(s,a)\Big|_{s=s_t, a=\mu(s_t)} \nabla_\theta \mu_\theta(s)\Big|_{s_t} \tag{26}$$

4.Perform soft updates on the parameters of the target Critic Network and target Actor Network, with the update formula as follows:

$$\begin{cases} \omega' \leftarrow \tau\omega + (1-\tau)\omega' \\ \theta' \leftarrow \tau\theta + (1-\tau)\theta' \end{cases} \tag{27}$$

with τ ∈ (0, 1).

5.Return to step 2 to continue learning at the next time step. When the reset conditions are met or the learning exceeds the set time steps, start a new episode.

3.2 Heading Controller Design Based on MPC Algorithm

Assuming the longitudinal velocity *u* of the USV remains constant, the linearized model of the USV can be written in state-space form, resulting in the system state-space equations:

$$\begin{aligned} \dot{\boldsymbol{x}}(t) &= \boldsymbol{A}\boldsymbol{x}(t) + \boldsymbol{B}\delta(t) \\ \boldsymbol{y}(t) &= \boldsymbol{C}\boldsymbol{x}(t) \end{aligned} \tag{28}$$

where x = v

output vector, and C is the output matrix. ? r ?T ψ represents the system state vector, δ is the input variable, that is, the thrust angle, A is the state transition ? matrix, ? B is the control input matrix, y = [ψ] is the system output vector, and C is the output matrix.

output variable, and C = 0 0 1 is the output matrix. Discretizing the continuous state-space equations yields the following:

$$\begin{aligned} \boldsymbol{x}_m(k+1) &= \boldsymbol{A}_m\boldsymbol{x}_m(k) + \boldsymbol{B}_m\boldsymbol{\delta}(k) \\ \boldsymbol{y}(k) &= \boldsymbol{C}_m\boldsymbol{x}_m(k) \end{aligned} \tag{29}$$

where $\boldsymbol{A}_m = (\boldsymbol{A}T + \boldsymbol{I})$, $\boldsymbol{B}_m = \boldsymbol{B}T$, $\boldsymbol{C}_m = \boldsymbol{C}$, and T represents the sampling period.

To address steady-state errors, incremental state-space equations are applied by sub-stituting state variables and control inputs with their respective changes. Differentiating both sides of the state-space equations results in the following:

$$\begin{aligned} \boldsymbol{x}_m(k+1) - \boldsymbol{x}_m(k) &= \boldsymbol{A}_m(\boldsymbol{x}_m(k) - \boldsymbol{x}_m(k-1)) + \boldsymbol{B}_m(\boldsymbol{\delta}(k) - \boldsymbol{\delta}(k-1)) \\ \boldsymbol{y}(k+1) - \boldsymbol{y}(k) &= \boldsymbol{C}_m(\boldsymbol{x}_m(k+1) - \boldsymbol{x}_m(k)) \end{aligned} \tag{30}$$

Define Δx m(k + 1) = xm(k + 1) − xm(k), Δδ(k) = δ(k) − δ(k − 1), and substitute them to obtain the incremental state-space equations:

$$\begin{aligned} &\begin{bmatrix} \Delta\boldsymbol{x}_m(k+1) \\ \boldsymbol{y}(k+1) \end{bmatrix} = \begin{bmatrix} \boldsymbol{A}_m & 0 \\ \boldsymbol{C}_m\boldsymbol{A}_m & 1 \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{x}_m(k) \\ \boldsymbol{y}(k) \end{bmatrix} + \begin{bmatrix} \boldsymbol{B}_m \\ \boldsymbol{C}_m\boldsymbol{B}_m \end{bmatrix} \Delta\boldsymbol{\delta}(k) \\ &\boldsymbol{y}(k) = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{x}_m(k) \\ \boldsymbol{y}(k) \end{bmatrix} \end{aligned} \tag{31}$$

The matrix form can be written as follows:

$$\begin{aligned} \boldsymbol{x}(k+1) &= \boldsymbol{A}_k\boldsymbol{x}(k) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) \\ \boldsymbol{y}(k) &= \boldsymbol{C}_k\boldsymbol{x}(k) \end{aligned} \tag{32}$$

If the control input sequence over the prediction horizon is available, future system outputs can be estimated using the system model and current state. Let N denote the prediction horizon. The equations for system state prediction are as follows:

$$\begin{cases} \boldsymbol{x}(k+1|k) = \boldsymbol{A}_k\boldsymbol{x}(k) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) \\ \boldsymbol{x}(k+2|k) = \boldsymbol{A}_k^2\boldsymbol{x}(k) + \boldsymbol{A}_k\boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k+1) \\ \cdots \\ \boldsymbol{x}(k+N|k) = \boldsymbol{A}_k^N\boldsymbol{x}(k) + \boldsymbol{A}_k^{N-1}\boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) + \ldots + \boldsymbol{A}_k\boldsymbol{B}_k\Delta\boldsymbol{\delta}(k+N-2) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k+N-1) \end{cases} \tag{33}$$

Substituting the output matrix Ck, the system output prediction equation is obtained as

follows:

$$\vec{y}_{k+1} = \boldsymbol{P}\boldsymbol{x}_k + \boldsymbol{H}\Delta\vec{\boldsymbol{u}}_k \tag{34}$$

where y k+1 represents the predicted system output sequence at the current time step, and → u k represents the current system input sequence.

$$\boldsymbol{P} = \begin{bmatrix} \boldsymbol{C}_k\boldsymbol{A}_k \\ \boldsymbol{C}_k\boldsymbol{A}_k^2 \\ \vdots \\ \boldsymbol{C}_k\boldsymbol{A}_k^n \end{bmatrix} \tag{35}$$

$$\boldsymbol{H} = \begin{bmatrix} \boldsymbol{C}_k\boldsymbol{B}_k & 0 & \dots & 0 \\ \boldsymbol{C}_k\boldsymbol{A}_k\boldsymbol{B}_k & \boldsymbol{C}_k\boldsymbol{B}_k & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{C}_k\boldsymbol{A}_k^{n-1}\boldsymbol{B}_k & \boldsymbol{C}_k\boldsymbol{A}_k^{n-2}\boldsymbol{B}_k & \dots & \boldsymbol{C}_k\boldsymbol{B}_k \end{bmatrix} \tag{36}$$

To achieve convergence of path-following errors while minimizing the variation in the thrust angle to reduce energy loss, the objective function for optimization is defined as follows:

$$\min J_k = \sum_{i=1}^{N} \|\boldsymbol{y}(k+i|k) - \psi_d\|_{\mathbf{Q}}^2 + \sum_{i=1}^{N} \|\Delta\boldsymbol{u}(k+i-1|k)\|_{\mathbf{R}}^2 \tag{37}$$

where N is the total number of optimization steps, Q = q1 ... qN ? represents the output error weight matrix, and R = r1 ... rN ?represents the input weight matrix.

In practice, there are thrust angle saturation limits and thrust angle rate saturation limits for the outboard motor. These constraints must be considered during the study. The two types of constraints can be expressed as follows:

$$\delta_{\min} \le \delta \le \delta_{\max} \tag{38}$$

$$\Delta\delta_{\min} \le \Delta\delta \le \Delta\delta_{\max} \tag{39}$$

Where δmin and δmax represent the minimum and maximum thrust angles of the outboard motor, while Δδmin and Δδmax represent the minimum and maximum thrust angle rates.

The optimization problem can be reformulated into a standard quadratic programming (QP) form, allowing the optimal thrust angle increment sequence to be determined. By adding the first element of this sequence to the previous thrust angle, the control input for the current time step is generated, and the process repeats. This approach, known as the rolling

optimization strategy, enables the desired heading angle control based on MPC.

外文译文

基于深度强化学习和模型预测控制算法的单舷外机无人水面艇路径跟踪技术研究

摘要

路径跟踪是无人水面艇（USV）的关键技术之一。本文提出了一种基于深度确定性策略梯度（DDPG）算法和模型预测控制（MPC）算法的单舷外机 USV 路径跟踪控制方法。首先，对 USV 的运动模型和舷外机模型进行了分析。随后，通过仿真和实船实验对所提出的 DDPG-MPC 方法与传统 ALOS-PID 方法进行了全面的性能比较。结果表明，在直线路径跟踪方面，DDPG-MPC 算法的平均横向误差和航向角误差分别比 ALOS-PID 算法降低了 37%和 21%。实船实验进一步验证了 DDPG-MPC 算法在实际环境中的优势。具体而言，在风浪流等干扰下，DDPG-MPC 算法的最大横向误差仅为 ALOS-PID 算法的三分之一。此外，DDPG-MPC 算法在长时间内保持了更高的纵向速度稳定性，而 ALOS-PID 算法则表现出更大的不稳定性和波动性。总体而言，研究结果证实了所提出方法的可行性和有效性，突显了其在提高单舷外机无人水面艇路径跟踪控制性能方面的潜力。

关键词：路径跟踪；无人水面艇；深度强化学习；深度确定性策略梯度；模型预测控制

1 简介

无人水面艇（USV）因其智能性、机动性和适应性，在诸如水域巡逻、水文调查、海上作业、救援任务以及军事用途等众多领域中越来越受到重视[1-4]。路径跟踪是无人水面艇自主导航的关键技术，其重点在于通过保持在可接受的误差范围内遵循规划路径来提高效率并确保安全，这与轨迹跟踪不同，轨迹跟踪要求在特定点严格遵守时间安排[5-7]。路径跟踪技术主要由路径引导和航向控制两部分组成[8-10]。在众多研究方法中，视线（LOS）和矢量场（VF）方法较为常见[11,12]。这些方法通过精心设计的引导法则，将路径跟踪难题转化为航向管理任务，从而有助于对欠驱动船舶进行控制。

多项进展提升了 LOS 方法的有效性。王等人[13]提出了一种基于预测器的固定时间 LOS 导引律，专门用于在未知干扰中航行的无人水面艇，具有全局固定时间收敛性，能够确保快速收敛。无论初始条件如何，都能实现稳定性和鲁棒性能。刘等人[14]开发了一种自适应线性导向（ALOS），能够精细地适应侧滑的变化，在各种条件下保持轨迹精度。周等人[15]在此基础上改进，提出了积分微分线性导向（IDLOS），该方法引入自适应元素以加快收敛速度并减少超调，从而优化响应时间和控制精度。进一步的进展来自童等人[16]，他们提出了一种有限时间、误差受限的线性导向方法，适用于在不确定情况下导航弯曲路径，扩大了线性导向的应用范围。此外，福森等人[17]设计了一种非线性自适应线性导向方法，能够有效应对环境漂移，如风和水流，在线性轨迹条件下确保半全局指数稳定性（USGES）。在对比分析中，ALOS 导向[16]在动态环境中因其快速适应性和稳健稳定性而脱颖而出，而 ALOS 法则[17]则提供有限时间收敛和横向误差约束，非常适合面临不可预测挑战的欠驱动系统。另一方面，王的基于预测器的固定时间 LOS 方法[13]提供了一个具有弹性的框架，确保了快速且精准的控制。然而，尽管这些先进的 LOS 方法复杂度更高且计算需求更大，但在需要即时响应和极高精度的场景中表现出色，ALOS 在资源有限的情况下尤其有效。洪等人[18]通过引入可变的“接受圆”半径来改进 LOS 方法，从而在航路点过渡期间增强了灵活性，这相较于传统方法有了显著的改进。相比之下，最初为微型飞行器开发的 VF 方法已被应用于无人水面艇[19-21]，对比研究表明这些方法在路径跟踪方面表现出色，但可能会导致偏航不稳定[22]。总体而言，这些改进在鲁棒性、收敛速度和计算效率之间取得了平衡，推动了无人船路径跟踪技术的发展。然而，该领域的当前研究仍存在局限性。基于视线法

（LOS 法）和虚拟力法（VF 法）的无人水面舰艇航向引导律的现有研究及实际应用存在固有缺陷[23]。视线法在前向距离设定时难以在响应时间和稳态误差之间取得平衡[24]。此外，在不同期望路径之间转换时，相关控制器的收敛速度无法得到保证[25]。相反，虚拟力法则存在水平方向上的舵机振荡问题[26]。

近期的研究[27-31]将深度强化学习新纳入无人水面艇（USV）的路径跟踪策略中，突显了控制方法论方面的最新进展。赵等人[6]开发了一种增强型深度 Q 网络（DQN），用于简化三自由度 USV 模型的控制，揭示了在海上导航场景中自适应学习的巨大潜力。钟等人[32]提出了一种新颖的控制方法，采用基于复合状态空间和动态奖励函数的深度确定性策略梯度（DDPG）算法，旨在减少大惯性和状态滞后对强化学习代理训练的影响。韩等人[33]开发了一个系统，该系统利用分布式深度强化学习和自适应神经网络实现 USV 的直线路径跟踪和编队控制，采用径向基函数神经网络（RBF NN）来逼近影响车辆的水动力和未知外部干扰，通过仿真结果表明其在跟踪控制方面具有很高的精度。此外，朱等人[34]提出了一种针对具有未知模型参数的欠驱动无人水面艇的智能路径跟踪控制策略，该策略结合了视线制导和 Q 学习，以提高在不确定动态下的路径跟踪精度。这些进展突显了将深度强化学习应用于实际问题时日益增长的灵活性和挑战性，展示了其在实现复杂海上导航任务方面的潜力，同时也凸显了计算复杂性、数据需求以及对动态环境变化响应时间等固有局限性。

然而，相当一部分研究仍依赖于配置在船尾的差速双螺旋桨推进器，无法直接转换为单舷外机布局[27-31]，这凸显了针对不同推进方式制定特定解决方案的必要性。

本文介绍了一种针对单舷外机无人水面艇（USV）的新型路径跟踪控制方法，该方法将深度强化学习与模型预测控制相结合。这种方法克服了传统直线航向（LOS）和矢量场（VF）方法的不足，专门针对单舷外机 USV 的特定推进特性进行了优化。通过在单舷外机 USV 任务中实施深度强化学习进行路径跟踪，我们显著提高了路径跟踪能力。同时，我们的强化学习方法 DDPG-MPC 还提高了基于强化学习的路径跟踪在复杂海洋环境中的通用性，为自主 USV 路径跟踪带来了创新。通过采用这种前沿策略，我们拓展了强化学习在不同操作条件下的实际应用，并提高了 USV 对更广泛环境挑战的适应性和恢复能力。

2 运动学建模

2.1 无人水面艇运动学建模

无人水面艇的运动学建模涉及推力器力和力矩、粘性流体动力学以及外部干扰等多种因素，具有高度非线性和多变的耦合特性，是一个非常复杂的过程[35]。建模时需要平衡两个方面。首先，为避免模型过于复杂而无法保证后续控制操作的实时性，建模过程中需要做出合理的假设和简化。其次，简化模型时要避免忽略一些关键的系统特性。

无人水面艇（USV）的运动学建模通常涉及两个坐标系，即世界坐标系 OXwYwZw 和固连于艇体的坐标系 OXbYbZb [36]，如图 1 所示。本文将世界坐标系设定为北东地（NED）坐标系。在此坐标系中，无人水面艇的六自由度（DOF）运动矢量可表示为 η = [x, y, z, φ, θ, ψ]T，其中 x、y 和 z 分别表示沿 Xw、Yw 和 Zw 轴的位移，φ、θ 和 ψ 分别表示绕 Xw、Yw 和 Zw 轴的横滚角、俯仰角和偏航角。

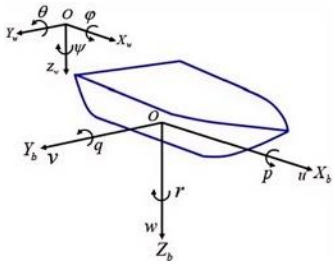

图 1 无人水面艇运动学建模的坐标系

在固连于船体的坐标系中，本文将原点 O 定义为无人水面艇（USV）的质心，Xb 轴指向艇的前进方向，Yb 轴指向艇的右舷方向，Zb 轴指向艇的下方向。绕 Xb、Yb 和 Zb 轴的旋转分别产生横摇角速度 p、纵摇角速度 q 和航向角速度 r。沿 Xb、Yb 和 Zb 轴的平移分别产生纵向速度 u、横向速度 v 和垂向速度 w。在固连于船体的坐标系中，USV 的速度可用向量 υ = [u, v, w, p, q, r]T 表示。

在无人水面艇（USV）动力学的研究中，更侧重于其平面运动状态。因此，横摇、纵摇和垂荡运动通常被忽略。于是，USV 的运动可以用基于拉格朗日力学的三自由度运动模型来表示，如方程（1）所示。

$$\boldsymbol{M}\dot{\boldsymbol{v}} + \boldsymbol{C}(\boldsymbol{v})\boldsymbol{v} + \boldsymbol{D}(\boldsymbol{v})\boldsymbol{v} = \boldsymbol{\tau} \tag{1}$$

其中，M 表示惯性矩阵，C（υ） 表示科里奥利力和离心力矩阵，D（υ） 表示

水动力阻尼矩阵。上述各项可通过以下公式计算得出：

$$\boldsymbol{M}=\begin{bmatrix} m-X_{\dot{u}} & 0 & 0 \\ 0 & m-Y_{\dot{v}} & mx_g-Y_{\dot{r}} \\ 0 & mx_g-Y_{\dot{r}} & I_z-N_{\dot{r}} \end{bmatrix} \tag{2}$$

$$\boldsymbol{C}(\boldsymbol{v})=\begin{bmatrix} 0 & 0 & -(m-Y_{\dot{v}})v-(mx_g-Y_{\dot{r}})r \\ 0 & 0 & (m-X_{\dot{u}})u \\ (m-Y_{\dot{v}})v+(mx_g-Y_{\dot{r}})r & -(m-X_{\dot{u}})u & 0 \end{bmatrix} \tag{3}$$

$$\boldsymbol{D}(\boldsymbol{v})=\begin{bmatrix} -X_u & 0 & 0 \\ 0 & -Y_v & -Y_r \\ 0 & -N_v & -N_r \end{bmatrix} \tag{4}$$

其中，Xu. 、Yv. 、Yr. 、Nr. 、Xu 、Yv 、Yr 、Nv 和 Nr 是无人水面艇的水动力系数。

υ 表示在附体坐标系中无人水面艇的三自由度速度矢量，如方程（5）所示。

$$\boldsymbol{v}=\begin{bmatrix} u & v & r \end{bmatrix}^T \tag{5}$$

τ 表示螺旋桨对无人水面艇施加的力，如方程（6）所示。

$$\boldsymbol{\tau}=\begin{bmatrix} \tau_u & \tau_v & \tau_r \end{bmatrix}^T \tag{6}$$

其中，τu 是纵向推力，τv 是侧向推力，τr 是偏航力矩。这三个量可以通过方程（7）计算得出。

$$\begin{cases} \tau_u=T\cos\delta \\ \tau_v=T\sin\delta \\ \tau_r=x_l T\sin\delta \end{cases} \tag{7}$$

其中，T 表示舷外机的推力，δ 表示舷外机推力的方向角，xl 表示舷外机旋转轴到无人水面艇旋转中心的距离。

仅由一台舷外发动机推进的无人水面艇（USV）只有两个控制变量：舷外发动机的推力角和推力，但无人水面艇需要控制三个自由度，因此属于欠驱动系统。基于单舷外发动机无人水面艇的欠驱动特性，在文献[37,38]中做出了如下定义：m11 = m - Xu. ，m22 = m - Yv. ，m23 = mxg - Yr. ，m32 = mxg - Nv. ，m33 = Iz - Nr. ，d11 = -Xu，d22 = -Yv，d23 = -Yr，d32 = -Nv，d33 = -Nr。因此，方程（1）可以转换为方程（8）。

$$\begin{bmatrix} m_{11}\dot{u} \\ m_{22}\dot{v}+m_{23}\dot{r} \\ m_{32}\dot{v}+m_{33}\dot{r} \end{bmatrix} + \begin{bmatrix} d_{11}u - m_{22}vr - m_{23}r^2 \\ d_{22}v - d_{23}r + m_{11}ur \\ m_{22}vu + m_{23}ru + d_{32}v - m_{11}uv + d_{33}r \end{bmatrix} = \begin{bmatrix} \tau_u \\ 0 \\ \tau_r \end{bmatrix} \tag{8}$$

假设船体的重心与几何中心重合，并且无人水面艇在纵向和横向对称，则有 xg = 0，Yr. = 0，Nv. = 0，Yr = 0 和 Nv = 0，从而方程（8）可简化为方程（9）。

$$\begin{cases} \dot{u} = \dfrac{m_{22}}{m_{11}}vr - \dfrac{d_{11}}{m_{11}}u + \dfrac{1}{m_{11}}\tau_u \\ \dot{v} = -\dfrac{m_{11}}{m_{22}}ur - \dfrac{d_{22}}{m_{22}}v \\ \dot{r} = \dfrac{m_{11}-m_{22}}{m_{33}}uv - \dfrac{d_{33}}{m_{33}}r + \dfrac{1}{m_{33}}\tau_r \end{cases} \tag{9}$$

2.2 舷外机运动学建模

从方程（7）可以看出，舷外机对无人水面艇施加的纵向推力、横向推力和转向力矩取决于舷外机的推力 T 和推力角 δ。因此，有必要对单个舷外机的推力模型和伺服模型进行建模。

2.2.1 舷外发动机模型

舷外发动机具有易于收起的优点，当两栖无人水面艇靠岸时，可将其收起，从而避免对地面造成干扰。这使其适合作为两栖无人水面艇的推进系统。舷外发动机产生的推力由螺旋桨的转速决定，其推力数学模型由公式（10）给出[39]：

$$T = (1 - t_P)\rho n^2 D_P^4 K_T(J_p) \tag{10}$$

其中，ρ 表示海水密度，n 表示螺旋桨转速，DP 表示螺旋桨直径。此外，tP 表示减速系数，KT（Jp）表示推力系数，Jp 表示进速系数。这些系数的计算方法在下文中有详细描述。

推进系数 tP 可以使用霍尔罗特经验公式[40]进行近似计算，该公式适用于单螺旋桨推进系统，如公式（11）所示。

$$t_P = 0.001979\frac{L}{(B - BC_{p1})} + 1.0585C_{10} - 0.000524 - 0.1418\frac{D_P^2}{Bd} + 0.0015C_{stern} \tag{11}$$

其中 L 表示水线长度，B 表示船宽，d 表示吃水深度。而 Cp1 可以通过公式（12）计算得出。

$$C_{p1} = 1.45C_P - 0.315 - 0.225L_{cb} \tag{12}$$

其中，CP 表示棱形系数，Lcb 表示浮力前半中心位置。

C10 表示与长宽比相关的系数，可利用公式（13）进行计算。Cstern 表示与船尾形状相关的系数，可利用公式（14）确定。

$$\begin{cases} C_{10} = B/L & L/B > 5.2 \\ C_{10} = 0.25 - \dfrac{0.003328402}{B/L - 0.134615385} & L/B \leq 5.2 \end{cases} \tag{13}$$

$$\begin{cases} C_{stern} = -10 \quad V - Shaped\ Hull \\ C_{stern} = 0 \quad Conventional\ Hull \\ C_{stern} = +10 \quad U - Shaped\ Hull\ with\ a\ Hogner\ Stern \end{cases} \tag{14}$$

推力系数 KT 可以通过多元回归分析计算得出[41]，如公式（15）所示。

$$K_T = \sum_{i=0}^{n1} \sum_{j=0}^{n2} A_{ij} \left(\frac{P}{D} \right)^i J_P^j \tag{15}$$

其中 JP 表示预进系数，wp 表示尾流系数，nDp

这可以通过 Bappmiller 公式来确定，如方程（16）所示。

$$w_p = 0.165 C_b^x \sqrt{\frac{\sqrt[3]{\nabla}}{D_p}} - \Delta w \tag{16}$$

其中，∇ 表示位移体积，x 是与螺旋桨安装位置相关的系数，如方程（17）所示，而Δw 表示尾流系数的修正值，可用方程（18）表示。

$$\begin{cases} x = 1 \quad Propeller\ Positioned\ on\ the\ Centerline \\ x = 2 \quad Propeller\ Positioned\ on\ the\ Side \end{cases} \tag{17}$$

$$\Delta w = \begin{cases} 0.1(F_n - 0.2), & F_n > 0.2 \\ 0, & F_n \leq 0.2 \end{cases} \tag{18}$$

其中，Fn = V/pgL 为弗劳德数，g 为重力加速度。

2.2.2 伺服模型

舷外发动机伺服模型可以表示为一个一阶惯性元件，这与 M. T. Meziou、J. Ghommam 和 N. Derbel 在其论文 [42] 中的方法一致，如方程（19）所示。

$$\dot{\delta} = -\frac{1}{T_d}\delta + \frac{1}{T_d}\delta_d \tag{19}$$

其中 Td 表示时间系数，δ 表示当前推进角，δd 表示目标推进角。

3 路径跟踪控制算法

本文提出了一种设计策略，将单舷外机无人水面艇（USV）的路径跟踪控制框架分为两个关键部分：路径跟踪制导律和航向控制。为克服传统算法的局限性，引入了一种将深度确定性策略梯度（DDPG）与模型预测控制（MPC）相结合的新方法，如图 2 所示。该方法从预先规划的路径中计算出期望航向角 ψd，并采用基于 MPC 的航向控制器来调整 USV 的航向角，从而实现有效的路径跟踪。

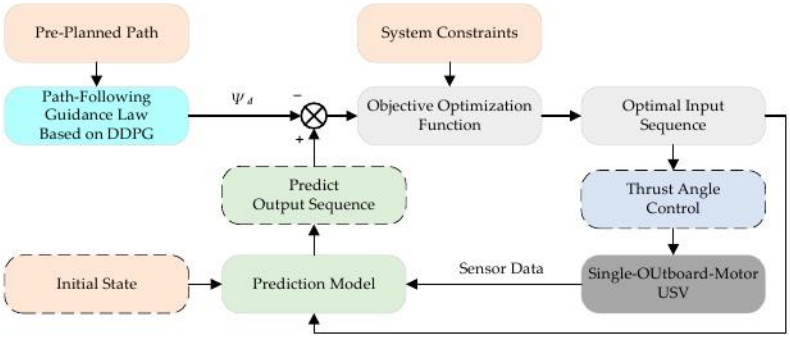

图 2 单舷外机无人水面艇路径跟踪控制算法框架

3.1 基于 DDPG 算法的路径跟踪制导律

3.1.1 马尔可夫决策过程的设计

在强化学习中，构建马尔可夫决策模型是关键要素。算法的效率和收敛性在很大程度上取决于对状态空间、动作空间和奖励函数的准确定义[43]。因此，要为无人水面艇（USV）的路径跟踪开发出高性能的制导律，设计一个适合单舷外机无人水面艇特定需求的马尔可夫决策模型至关重要。

状态空间定义如下：

$$S = [u, r, \beta, \dot{\beta}, \varepsilon, \dot{\varepsilon}, \Delta\psi_d^p] \tag{20}$$

其中，u 表示无人水面艇的纵向速度，r 表示无人水面艇的俯仰角速度，β 是无人水面艇航向角与期望路径角之间的偏差，ε 是期望路径与无人水面艇实时位置之间的横向误差，而 Δψd 表示 p 在上一时间步所采取的行动，即无人水面艇（USV）航向角的期望变化。

在路径跟踪控制中，通常更注重无人水面艇（USV）的纵向速度，而横向漂移速度往往被忽视。为了确保路径跟踪制导律能够根据 USV 的纵向速度 u 实时生成最优期望航向角，本研究将纵向速度纳入状态空间。状态空间包括当前航向角与期望路径角之

间的偏差β，以及期望路径与 USV 实时位置之间的横向误差，反映实际位置与期望位置之间的差异。此外，为了评估 USV 状态的变化是否符合减少跟踪误差的趋势，状态空间还整合了航向与路径偏差的变化率。

角度 β、横向误差的变化率以及在上一时间步长所采取的行动 Δψd p 。

动作空间定义如下：

$$\boldsymbol{A} = [\Delta\psi_d] \tag{21}$$

Δψd 表示无人水面艇（USV）期望航向角的变化量，而 Δψd ∈ [−π，π] 使得 USV 的期望航向角 ψd 可通过公式 ψd = ψ + Δψd 计算得出，其中 ψ 表示 USV 的实际航向角。

奖励函数定义如下：

$$r = \begin{cases} r_\varepsilon + r_\beta + r_{\Delta\psi_d} & |\beta| < \frac{\pi}{2} \\ r_\beta & |\beta| \geq \frac{\pi}{2} \end{cases} \tag{22}$$

每个术语的定义如下：

$$r_\varepsilon = e^{-k_\varepsilon|\varepsilon|} \tag{23}$$

$$r_\beta = e^{-k_\beta|\beta|} \tag{24}$$

$$r_{\Delta\psi_d} = e^{-k_{\Delta\psi_d}|\Delta\psi_d|} \tag{25}$$

其中 kε、kβ 和 kΔψd 是通过多次实验校准的常数，且 kε= = 0.7、 kβ= = 4.82、kΔψd = 0.3 。

公式（23）描述了奖励函数 r ε for 与无人船实际位置和期望路径之间的横向误差 ε 的关系。如图 3a 所示，该函数具有以下特点：当横向误差 ε 为 0 时，奖励值 r ε reaches 为 1；随着 | ε | 值的增加，奖励值 r ε decreases 迅速下降。然而，当误差 | ε | 增加到 8 米（约为一艘船的长度）时，奖励值 r ε is 几乎接近 0。这种设计以指数函数的形式鼓励智能体在与环境交互过程中积极减小横向误差，从而实现更精确的路径跟踪。

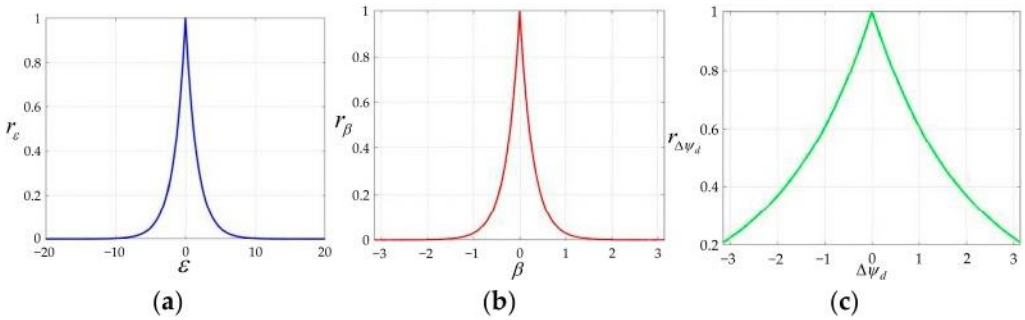

图 3. 奖励函数。（a）与无人水面艇实际位置和期望跟踪路径之间的横向误差 ε 相关的奖励函数 rεrelated 。（b）与无人水面艇航向角和期望路径角偏差 βbetween 相关的奖励函数 rβrelated 。（c）与无人水面艇期望航向角变化 Δψd 相关的奖励函数 rΔψd 。

公式（24）是针对无人水面艇（USV）航向角与期望路径角偏差β的奖励函数 rβrelated 。该函数的图像如图 3b 所示。当角度偏差β为 0 时，其值达到最大值 1。为防止无人水面艇偏离期望路径，即当 |β|> π2 时，其值接近 0。

公式（25）定义了与无人水面艇（USV）期望航向角变化相关的奖励函数 rΔψd 。如图 3c 所示，该函数经过精心设计，旨在减少期望航向方向的突然改变。通过这种方式，它确保了调整过程的平滑和连续，提高了 USV 导航路径的稳定性和精度。这种方法最终有助于实现更高效、更可靠的路径跟踪。

方程（22）是经过加权处理后的个体奖励的最终奖励函数 r 。

当航向角与期望路径角的偏差较大时，应尽快调整航向角，以防止无人水面艇的实际位置与期望跟踪路径之间的偏差进一步增大。因此，此时选择 r = rβis；当 |β| < π2 时，考虑所有三个奖励函数，选择 r = rε+ rΔψd + rβis 。

3.1.2 DDPG 算法的实现

DDPG 算法是一种基于演员-评论家框架的深度强化学习方法[44]，其结构原理如图 4 所示。

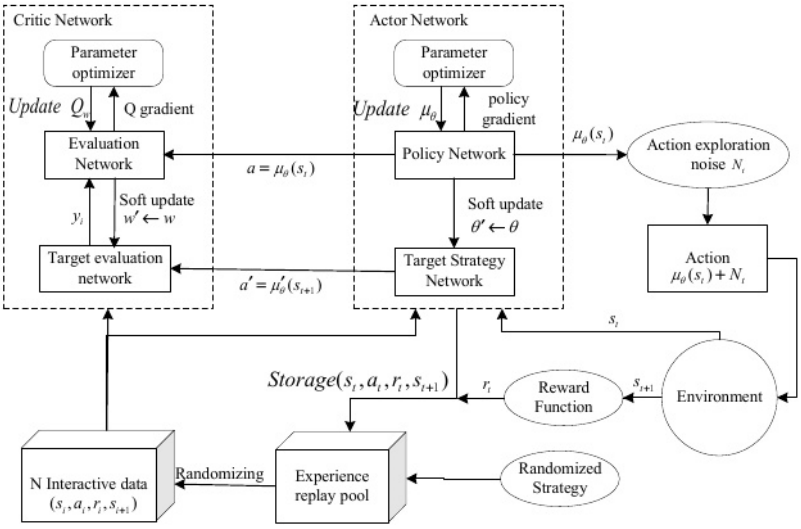

图 4 DDPG 算法框架

DDPG 智能体由策略网络和价值网络组成。策略网络，即所谓的“行为者网络”，以状态向量 st 作为输入，并根据行动策略生成行动向量 μθ(st)，其中 θ 表示行为者网络的权重。从结构上看，行为者网络包含一个输入层、两个隐藏层和一个输出层。输入层的大小与状态向量的维度一致，而输出层则与行动向量的维度相匹配。隐藏层使用 ReLU 作为激活函数，输出层则采用 tanh 函数对行动值进行归一化处理，随后将其缩放至实际的行动范围。

价值网络，即所谓的“评价网络”，以由“行动者网络”生成的状态向量 st 和行动向量 at 作为输入，并输出行动价值 Qω(st， at)，其中 ω 代表评价网络的权重。状态向量通过两个隐藏层进行处理，而行动向量则通过一个隐藏层。这些输出随后通过张量加法进行组合，接着再经过一个隐藏层和一个输出层。所有隐藏层均采用 ReLU 作为激活函数。DDPG 代理的训练过程包括以下步骤：

1.随机初始化策略网络 μ（s|θ） 和价值网络 Q（s, a|ω）；初始化目标网络的策略网络 ω‘←ω 和 θ’←θ；初始化经验回放缓冲区 R。

2.通过在环境中随机初始化无人水面艇（USV）的状态作为初始状态 s1 来开启新的回合。在每个时间步长，当前状态向量 st 和动作策略 μθ(st) 与探索噪声 Nt 相结合，生成动作向量 at = μθ(st) + Nt，该向量定义了 USV 的期望航向角。航向控制器随后跟踪此角度，环境提供更新后的状态向量 st+1。同时，利用环境数据（如横向误差和航向角误差）计算奖励。在每个时间步长生成的交互数据（st, at, rt, st+1）被保存在经验回放缓冲区 R 中。

3.从经验回放缓冲区 R 中随机抽取 N 个交互数据来更新网络。通过最小化损失函数 L = N1 ∑i (yi −Qω(si, ai))2 来更新评论家网络，其中 yi = ri + γQ 'ω'（si+1， μ‘θ’（si+1）） 表示目标评论家网络输出的目标值。根据采样的梯度更新演员网络，其梯度公式如下：

$$\nabla_\theta \boldsymbol{\mu}|_{s_t} \approx \frac{1}{N}\sum_i \nabla_{\boldsymbol{a}} Q_\omega(\boldsymbol{s},\boldsymbol{a})\Big|_{\boldsymbol{s}=\boldsymbol{s}_t,\boldsymbol{a}=\boldsymbol{\mu}(\boldsymbol{s}_t)} \nabla_\theta \boldsymbol{\mu}_\theta(\boldsymbol{s})\Big|_{\boldsymbol{s}_t} \tag{26}$$

4.对目标评论家网络和目标行动者网络的参数执行软更新，更新公式如下：

$$\begin{cases} \omega' \leftarrow \tau\omega + (1-\tau)\omega' \\ \theta' \leftarrow \tau\theta + (1-\tau)\theta' \end{cases} \tag{27}$$

其中 τ ∈(0, 1) 。

5.返回步骤 2，在下一个时间步继续学习。当满足重置条件或学习超过设定的时间步时，开始新的回合。

3.2 基于 MPC 算法的航向控制器设计

假设无人水面航行器（USV）的纵向速度 u 保持恒定，则 USV 的线性化模型可以写成状态空间形式，从而得到系统状态空间方程：

$$\begin{aligned} \dot{\boldsymbol{x}}(t) &= \boldsymbol{A}\boldsymbol{x}(t) + \boldsymbol{B}\boldsymbol{\delta}(t) \\ \boldsymbol{y}(t) &= \boldsymbol{C}\boldsymbol{x}(t) \end{aligned} \tag{28}$$

其中 x = v ψ 是系统状态向量，δ 是输入变量，即推力角，A 是状态转移矩阵，B 是控制输入矩阵，y = [ψ] 是系统的输出。

输出变量，且 C = 0 0 1 为输出矩阵。对连续状态空间方程进行离散化可得以下结果：

$$\begin{aligned} \boldsymbol{x}_m(k+1) &= \boldsymbol{A}_m\boldsymbol{x}_m(k) + \boldsymbol{B}_m\boldsymbol{\delta}(k) \\ \boldsymbol{y}(k) &= \boldsymbol{C}_m\boldsymbol{x}_m(k) \end{aligned} \tag{29}$$

其中 Am = (AT + I)，Bm = BT，Cm = C，T 表示采样周期。

为解决稳态误差问题，通过将状态变量和控制输入替换为其各自的增量，应用增量状态空间方程。对状态空间方程两边求导可得以下结果：

$$\begin{aligned} \boldsymbol{x}_m(k+1) - \boldsymbol{x}_m(k) &= \boldsymbol{A}_m(\boldsymbol{x}_m(k) - \boldsymbol{x}_m(k-1)) + \boldsymbol{B}_m(\boldsymbol{\delta}(k) - \boldsymbol{\delta}(k-1)) \\ \boldsymbol{y}(k+1) - \boldsymbol{y}(k) &= \boldsymbol{C}_m(\boldsymbol{x}_m(k+1) - \boldsymbol{x}_m(k)) \end{aligned} \tag{30}$$

定义增量 Δx m(k + 1) = xm(k + 1) - xm(k)，增量 Δδ(k) = δ(k) - δ(k - 1)，并代入以得到增量状态空间方程：

$$\begin{aligned}
&\begin{bmatrix} \Delta \boldsymbol{x}_m(k+1) \\ \boldsymbol{y}(k+1) \end{bmatrix} = \begin{bmatrix} \boldsymbol{A}_m & 0 \\ \boldsymbol{C}_m\boldsymbol{A}_m & 1 \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{x}_m(k) \\ \boldsymbol{y}(k) \end{bmatrix} + \begin{bmatrix} \boldsymbol{B}_m \\ \boldsymbol{C}_m\boldsymbol{B}_m \end{bmatrix} \Delta\boldsymbol{\delta}(k) \\
&\boldsymbol{y}(k) = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{x}_m(k) \\ \boldsymbol{y}(k) \end{bmatrix}
\end{aligned} \tag{31}$$

矩阵形式可写成如下形式：

$$\begin{aligned}
&\boldsymbol{x}(k+1) = \boldsymbol{A}_k\boldsymbol{x}(k) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) \\
&\boldsymbol{y}(k) = \boldsymbol{C}_k\boldsymbol{x}(k)
\end{aligned} \tag{32}$$

如果在预测时域内控制输入序列是已知的，那么利用系统模型和当前状态就可以估计出未来的系统输出。设 N 表示预测时域。系统状态预测的方程如下：

$$\begin{cases}
\boldsymbol{x}(k+1|k) = \boldsymbol{A}_k\boldsymbol{x}(k) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) \\
\boldsymbol{x}(k+2|k) = \boldsymbol{A}_k^2\boldsymbol{x}(k) + \boldsymbol{A}_k\boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k+1) \\
\dots \\
\boldsymbol{x}(k+N|k) = \boldsymbol{A}_k^N\boldsymbol{x}(k) + \boldsymbol{A}_k^{N-1}\boldsymbol{B}_k\Delta\boldsymbol{\delta}(k) + \ldots + \boldsymbol{A}_k\boldsymbol{B}_k\Delta\boldsymbol{\delta}(k+N-2) + \boldsymbol{B}_k\Delta\boldsymbol{\delta}(k+N-1)
\end{cases} \tag{33}$$

将输出矩阵 Ck 代入，可得系统输出预测方程如下：

$$\vec{\boldsymbol{y}}_{k+1} = \boldsymbol{P}\boldsymbol{x}_k + \boldsymbol{H}\Delta\vec{\boldsymbol{u}}_k \tag{34}$$

其中，y_{k+1} 表示当前时间步的预测系统输出序列，且 u(k) 表示当前系统的输入序列。

$$\boldsymbol{P} = \begin{bmatrix} \boldsymbol{C}_k\boldsymbol{A}_k \\ \boldsymbol{C}_k\boldsymbol{A}_k^2 \\ \vdots \\ \boldsymbol{C}_k\boldsymbol{A}_k^n \end{bmatrix} \tag{35}$$

$$\boldsymbol{H} = \begin{bmatrix} \boldsymbol{C}_k\boldsymbol{B}_k & 0 & \dots & 0 \\ \boldsymbol{C}_k\boldsymbol{A}_k\boldsymbol{B}_k & \boldsymbol{C}_k\boldsymbol{B}_k & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{C}_k\boldsymbol{A}_k^{n-1}\boldsymbol{B}_k & \boldsymbol{C}_k\boldsymbol{A}_k^{n-2}\boldsymbol{B}_k & \dots & \boldsymbol{C}_k\boldsymbol{B}_k \end{bmatrix} \tag{36}$$

为了在最小化推力角变化以减少能量损失的同时实现路径跟踪误差的收敛，优化的目标函数定义如下：

$$\min J_k = \sum_{i=1}^{N} \|\boldsymbol{y}(k+i|k) - \psi_d\|_{\mathrm{Q}}^2 + \sum_{i=1}^{N} \|\Delta\boldsymbol{u}(k+i-1|k)\|_{\boldsymbol{R}}^2 \tag{37}$$

其中 N 表示优化步骤的总数，Q = q1 ... qN？表示输出误差权重矩阵，R = r1 ... rN？表示输入权重矩阵。

实际上，舷外机存在推力角饱和限制和推力角速率饱和限制。在研究过程中必须考虑这些限制。这两种限制可表述如下：

$$\delta_{\min} \le \delta \le \delta_{\max} \tag{38}$$

$$\Delta\delta_{\min} \leq \Delta\delta \leq \Delta\delta_{\max} \tag{39}$$

其中，δmin 和 δmax 分别表示外侧发动机的最小和最大推力角，而 Δδmin 和 Δδmax 则分别表示最小和最大推力角速率。

该优化问题可以重新表述为标准二次规划（QP）形式，从而确定出最优推力角增量序列。将该序列的第一个元素加到前一个推力角上，即可生成当前时间步的控制输入，然后重复此过程。这种被称为滚动优化策略的方法，能够基于模型预测控制实现期望航向角控制。